

Surface Effects for Deterministic Parallelism

Vitor Rodrigues*

Matthew Fluet†

Abstract

Static effect systems can guarantee deterministic parallelism, but region-level approximations often reject computations whose concrete run-time accesses are independent. Dynamic and optimistic systems recover more parallelism by observing concrete accesses, but typically require logging, validation, rollback, or replay. This paper explores a middle ground: *surface-effect terms*, source-level executable effect terms that are statically checked against the computations they summarize and evaluated before those computations are run.

We present a region-polymorphic simply typed λ -calculus with mutable references and executable surface effects. The mechanized development proves that verified surface effects cover the dynamic traces of the computations they summarize, and that terminating continuation-machine executions are deterministic. It also validates an arbitrary scheduler relation for successful checked pair-parallel joins. Surface effects therefore provide a statically verified, source-level form of pre-execution access prediction: programmers may expose value-dependent access information before a computation is run, while the metatheory ensures that the surface effect covers the computation trace it claims to summarize.

Introduction

Multicore processors make parallelism increasingly attractive, but shared-state parallel programs are difficult to reason about. The programmer wants the performance benefits of running independent computations at the same time, while the language designer wants the semantic simplicity of deterministic execution: well-typed programs should not acquire data races, deadlocks, or schedule-dependent results merely because the implementation exploits parallel hardware. A central challenge is therefore to expose parallelism without asking the programmer to reason directly about low-level interleavings.

The language studied in this paper is a region-polymorphic λ -calculus with mutable references and executable surface effects. Typability is established using a type-and-effect discipline in the Lucassen–Gifford lineage (Lucassen and Gifford, 1988). Region effects and non-interference checks for deterministic parallelism are not, by themselves, the novelty claimed here; DPJ, for example, uses modular compile-time type-and-effect checking to guarantee deterministic semantics (Bocchino et al., 2009). The question we ask is more specific: can the programmer provide a checked, executable surface effect that is more precise than a coarse static region effect, but less intrusive than fully logging or validating every dynamic access?

Throughout the paper, we call the source-level executable effect-description terms *surface-effect terms*; when the context is clear, we use *surface effects* as shorthand for this term language. Evaluating a surface-effect term yields a *computed effect*, which is the run-time effect value available to later analyses or runtimes. The contribution is not merely that effects appear in types, but that a computation can be summarized by another well-typed source term that computes a surface effect for that computation before the computation itself is run.

Two degenerate surface-effect terms are useful for intuition. ‘Pure’ denotes the empty surface effect and is appropriate for computations that perform no dynamic memory actions. ‘Top’ is the opposite extreme: it is a sound surface effect for any computation, but it gives up precision and therefore prevents a useful address-level comparison. Between these extremes, surface-effect terms can compute value-dependent effects such as concrete reads and writes. This precision is useful when two computations mention the same static region but access different concrete locations within that region.

*VORTEX-Colab, Porto, Portugal. vitor.rodrigues@vortex-colab.com

†Rochester Institute of Technology, New York, USA. mtf@cs.rit.edu

The central intuition is that one well-typed term is summarized by *another* well-typed term of the same language. A surface effect may inspect run-time values and, for example, compute a concrete region/location pair instead of retaining only a region-level read or write. Such a surface effect may distinguish accesses that a region-only static effect merges. The formal development does not define a global precision lattice relating static effects, computed effects, and traces, nor does it prove that a more precise surface effect is necessarily more expensive. The soundness claim is instead expressed by explicit approximation relations: static effects soundly cover dynamic actions, and a verified surface-effect term evaluates to a computed effect that soundly covers the trace of the computation it summarizes.

Put simply, the surface-effects approach is a hybrid design point for value-dependent effect prediction. Purely static approaches are modular and perform the conflict check at compile time, but they may be conservative when pointer-rich data structures collapse many concrete locations into the same abstract region. Purely dynamic and optimistic approaches observe concrete accesses directly, and hence can be very precise, but they usually require logging, validation, rollback, or replay. Surface effects place the check in between: a source-level surface effect is supplied and verified statically, evaluated before the summarized computation runs, and then used as a more precise dynamic description of that computation’s future accesses.

Figure 1 previews this design point symbolically. The notation is introduced fully in Sections 2–4. Here it is only schematic: $s \longrightarrow s'; \eta$ denotes one step of the operational machine together with the trace fragment η emitted by that step, while $s \longrightarrow^* s'; \phi$ denotes a finite execution whose fragments have been accumulated into the machine trace ϕ . Structured trace views are introduced later as auxiliary views over this list trace. Static checking compares region effects ϵ_i . Surface-effect checking first verifies source expressions against surface-effect terms $\hat{\Theta}_i$, then evaluates those terms by the same operational machine to computed effects Θ_i . Dynamic approaches compare traces ϕ_i obtained from the actual executions. In the figure, $e_i \rightsquigarrow \phi_i$ is a trace-only abbreviation of the standard finite-execution relation, with heaps, environments, values, and continuations suppressed. The table uses level-specific conflict symbols: $\#$ for static region effects, $\otimes$ for computed effects Θ , and $\boxtimes$ for action/trace-level conflicts. In the pair-parallel machine the computed-effect comparison is implemented by a decidable precheck, followed by a defensive trace-disjointness check after the branch traces have been produced.

Static effects are inexpensive and available at compile time, but their region-level approximation may reject independent concrete accesses. Surface effects pay the cost of evaluating executable surface effects before running the summarized computations, but can recover value-dependent precision. Dynamic tracking observes the concrete accesses themselves, but usually requires logging, validation, rollback, or replay.

Three ways to compare effects			
Static region effects	$\Gamma \vdash e_1 : \tau_1 ! \epsilon_1$	$\epsilon_1 \# \epsilon_2$	$\Gamma \vdash e_2 : \tau_2 ! \epsilon_2$
Surface effects	$\Gamma; \Delta; P \vdash e_1 \blacktriangleleft \hat{\Theta}_1$		$\Gamma; \Delta; P \vdash e_2 \blacktriangleleft \hat{\Theta}_2$
and computed effects	$\hat{\Theta}_1 \longrightarrow^* \Theta_1$ $\phi_1 \sqsubseteq \Theta_1$	$\Theta_1 \otimes \Theta_2$	$\hat{\Theta}_2 \longrightarrow^* \Theta_2$ $\phi_2 \sqsubseteq \Theta_2$
Dynamic traces	$e_1 \rightsquigarrow \phi_1$	$\phi_1 \boxtimes \phi_2$	$e_2 \rightsquigarrow \phi_2$

Figure 1: Surface effects as a middle ground between static region effects and dynamic access tracking.

Operationally, the ordinary continuation-machine relation $s \longrightarrow s'; \eta$ evaluates expression structure and emits the trace fragment η . A surface-effect term is evaluated by this same machine; there is no separate surface-effect evaluator. Thus the paper’s runtime theorem is an operational theorem about finite small-step executions, not merely a statement about typing rules.

The static semantics uses type-and-effect judgments of the form $\Gamma \vdash e : \tau ! \epsilon$, where Γ is a typing context, e is an expression, τ is a type, and ϵ is a static effect (see Section 5). The static semantics of surface effects defines the back-triangle relation ($\blacktriangleleft$), which states that a computational expression e is correctly summarized by a surface-effect term $\hat{\Theta}$. At run time, that term is evaluated by the same small-step machine as every other expression. When it returns an effect value, that value is the computed effect Θ . If a computed effect is too imprecise, for example because a surface effect evaluated to $\top$, the result remains sound but conservative.

Static region/effect systems such as DPJ make non-interference a modular compile-time obliga-

tion, but region-level effects may merge concrete accesses that are distinct at run time (Bocchino et al., 2009). Runtime inspector-executor techniques address irregular dependences by discovering conflict-free work before executing it in parallel (Arenaz et al., 2004). Surface effects occupy a related middle position: the access description is supplied as a typed source-language term, checked against the computation, and evaluated before the summarized computation is run. Unlike optimistic post-validation, the calculus does not start the summarized computation and then roll it back after discovering a surface-effect conflict; however, this fact alone does not establish that pre-checking is cheaper in total.

The formal contribution is therefore a sound scheduling interface rather than an asymptotic performance result. At compile time, an inductive correctness relation validates programmer-written computation/surface-effect pairs. At run time, the surface-effect terms evaluate to computed effects that may contain concrete region/location information. The formal development connects these stages: branch correctness gives trace approximation, the pair-parallel checks reject conflicting computed effects or conflicting branch traces, and terminal continuation-machine theorems connect terminating traces to unique final heaps. For the pair-parallel run phase, the metatheory also includes an arbitrary scheduler whose successful checked joins are deterministic; the ordinary left-then-right machine run embeds as one scheduler instance. False positives remain possible because surface effects are approximations and ‘Top’ is deliberately maximally conservative.

The contributions of this paper are:

- a region-polymorphic λ -calculus with mutable references and surface-effect terms as ordinary expressions returning computed effects, together with a trace-emitting continuation-machine semantics that evaluates λ -terms and surface-effect terms uniformly;
- an inductive correctness relation for statically verifying that a programmer-written, value-dependent surface effect covers the computation it summarizes;
- a checked pair-parallel interface based on a decidable computed-effect precheck and a final trace-disjointness check, connecting verified branch surface effects to deterministic terminal executions and to a deterministic scheduler relation for successful checked joins;
- mechanized soundness theorems: progress, store-resolved preservation, static-effect trace soundness, terminal determinism, and terminal surface-effect correctness for checked continuation-machine executions.

The rest of this paper is organized as follows. We start by presenting a motivating example in Section 1. Section 2 distinguishes static effects, dynamic traces, surface-effect terms, and computed effects. The syntax for types and the surface language, including the surface-effect language, is given in Section 3. The static semantics of the type-and-effect system is presented in Section 5, and the operational semantics of the ordinary continuation machine is presented in Section 4. The computed-effect conflict and disjointness are introduced in Section 2 before presenting the correctness theorem for surface-effect terms in Section 6. The small-step finite-prefix safety layer is summarized in Section 7. We conclude in Section 9 after discussion of related work.

1 Motivating Example

The ML-like program in Fig. 2 illustrates the intuition behind surface-effect terms and checked pair-parallel execution. The function `incr` conditionally increments the integer stored in a mutable reference. The type assigned to `incr` is, in lightweight effect-arrow notation,

$$\text{incr} : \forall \rho. \text{ref } \rho \text{ int} \xrightarrow{\{\mathbf{R}(\rho), \mathbf{W}(\rho)\}} \text{unit}.$$

The latent static effect $\{\mathbf{R}(\rho), \mathbf{W}(\rho)\}$ says that the reference may be read and written. A conventional region-effect check therefore treats two calls to `incr` at the same region as potentially conflicting. Write `eff` for the type of computed-effect values. The corresponding executable surface-effect function has type

$$\text{incr_eff} : \forall \rho. \text{ref } \rho \text{ int} \xrightarrow{\emptyset} \text{eff},$$

because evaluating the summary expression constructs an effect value without performing the update summarized by `incr_eff`.

This is conservative for homogeneous collections. In `loop`, every reference in the input list lives in the same concrete region, so two variables `x` and `y` drawn from the list have the same region-level type. Static region effects cannot distinguish whether they are aliases or two different locations inside that region. Surface effects recover that concrete-address information before the summarized computations are run.

```
fun incr [ρ] (r : int ref at ρ) : unit =
  if isPrime (!r) then r := !r + 1 else ()

fun incr_eff [ρ] (r : int ref at ρ) : eff =
  Read(r) ⊕ Write(r)

fun loop (xs : int ref at 7 list) : unit =
  case xs of
  [] => ()
  | x :: [] => incr[7] x
  | x :: y :: zs =>
    let val () =
      (incr[7] x with incr_eff[7] x,
       incr[7] y with incr_eff[7] y)
    in loop zs end
```

Figure 2: Motivating example: pairwise checked calls over a homogeneous list.

The recursive case of `loop` uses the checked pair-parallel notation $\langle \dots, \dots \rangle$ to process two list elements at a time. The `with` annotation in the listing is expository: in the formal syntax, the computational and surface-effect bodies are packaged together in the same recursive abstraction. Thus the pair-parallel machine can determine the corresponding surface-effect applications from the two computational applications, evaluate those surface effects first, and then decide whether the parallel run phase is allowed.

The role of `incr_eff` is to compute the effect that the static region effect cannot express. The surface-effect term $\widehat{\text{Read}(r)} \oplus \widehat{\text{Write}(r)}$ is evaluated as an ordinary effect-valued expression. On list element `x`, it can produce a computed effect such as

$$\Theta_x = \{\text{Read}(l_x @ r), \text{Write}(l_x @ r)\},$$

where $l_x @ r$ is the concrete address stored in `x`; similarly `incr_eff[7] y` produces Θ_y for `y`. If $l_x \neq l_y$, the computed effects can pass the pair-parallel precheck even though both computations have the same static region effect. The loop can then uncover pairwise parallel work that a region-only check would reject. If $l_x = l_y$, the computed effects conflict: the ordinary machine does not enter the parallel run phase and falls back to sequential execution, while checked execution reports the failed precheck. After a successful precheck, the machine also checks the produced branch traces for dynamic conflicts before returning the pair value.

The point of the example is not that every invocation of `incr` is safe to parallelize. Statically, each call has the region effect $\{R(\rho), W(\rho)\}$, so two calls at the same region appear to conflict. The surface-effect term `incr_eff` refines that coarse description by computing the effect $\widehat{\text{Read}(r)} \oplus \widehat{\text{Write}(r)}$ from the actual argument `r`. The computed effect can mention concrete region/location pairs rather than only the region, which is exactly the information needed by the recursive pair-parallel step in `loop`.

The same example also illustrates the precision spectrum. A pure computation may be summarized by ‘Pure’, the empty surface-effect term. The computation `incr` cannot soundly use ‘Pure’, because it reads and writes its argument. It could, however, always use ‘Top’: this surface effect is sound but maximally conservative, so any later consumer of the computed effect would learn no useful address-level information. The useful surface effect lies between these extremes: $\widehat{\text{Read}(r)} \oplus \widehat{\text{Write}(r)}$ is still sound, but it exposes enough run-time address information to avoid some false conflicts in the list traversal.

We write $c \blacktriangleleft s$ to mean that the surface-effect term s summarizes the computation c . Thus, if `incr_eff` is a correct summary for `incr`, the judgment $\text{incr} \blacktriangleleft \text{incr_eff}$ must hold. The precision slogan is

$$\epsilon \supseteq \Theta \supseteq \phi,$$

but this is not a single global order. It abbreviates two typed bridges: a static effect ϵ covers the region erasure of a computed effect Θ , and the computed effect Θ covers the dynamic trace ϕ by the relation $\phi \sqsubseteq \Theta$. In the motivating case, the programmer-written surface-effect term is checked once as part of the program, but its value-dependent result is computed at run time by the same machine and used by the pair-parallel check.

Finally, any surface-effect term checked as a summary for a computation is required to be heap-neutral: evaluating it may read, but it must not allocate or write. The purpose of `incr_eff` is to inspect the reference value well enough to construct a computed effect, not to perform the update that `incr` will perform. This restriction is what lets the metatheory separate surface-effect evaluation from the computation it summarizes: evaluating the surface effect can affect the scheduling decision, but it cannot itself perform the heap updates whose safety is being checked.

2 Static Effects, Dynamic Traces, Surface Effects, and Computed Effects

This section fixes the vocabulary that is only previewed in the Introduction. The development uses four related, but deliberately distinct, notions of “effect”: static effects, dynamic traces, surface-effect terms, and computed effects. The distinction is essential. Static effects are compile-time approximations; dynamic traces are observations of execution; surface-effect terms are source syntax; and computed effects are the values obtained by running that syntax.

Regions, addresses, and actions. We write ρ for a region variable, r for a run-time region constant, l for a heap location, and $l@r$ for the concrete address consisting of location l in region r . A static region p may be either a region variable or a region constant. Static actions describe allocation, read, and write at the level of regions, whereas dynamic actions describe concrete events at allocated addresses. In the continuation-machine development, dynamic actions record the region and location of the event; heap values remain in the machine state rather than in the trace label.

Effect Definitions

$\rho \in \text{RgnVar}$	region variables
$r \in \text{RgnConst}$	region constants
$l \in \text{Loc}$	heap locations
$l@r \in \text{Addr} \triangleq \text{Loc} \times \text{RgnConst}$	concrete heap addresses
$p \triangleq \rho \mid r$	static regions
$\alpha \triangleq \text{A}(p) \mid \text{R}(p) \mid \text{W}(p)$	static actions
$\epsilon \in \mathcal{P}(\alpha)$	static effects
$\iota \triangleq \text{Alloc}(l@r) \mid \text{Read}(l@r) \mid \text{Write}(l@r)$	dynamic actions
$\eta \triangleq \text{Nil} \mid \iota$	one-step trace fragments
$\phi \triangleq [] \mid \iota :: \phi$	machine traces
$\Phi \triangleq \eta \mid \Phi_1 \bullet \Phi_2 \mid \Phi_1 \parallel \Phi_2$	structured trace views
$\theta \triangleq \widehat{\text{A}}(r) \mid \widehat{\text{R}}(r) \mid \widehat{\text{W}}(r) \mid \widehat{\text{Alloc}}(l@r) \mid \widehat{\text{Read}}(l@r) \mid \widehat{\text{Write}}(l@r)$	computed actions
$\Theta \triangleq \top \mid \{\theta_1, \dots, \theta_n\}$	computed effects

Static effects and dynamic traces. A static effect ϵ is the effect component of a type-and-effect judgment, defined in Section 5. It abstracts over all possible executions of the typed expression using region-level actions such as $\text{R}(\rho)$ or $\text{W}(\rho)$. A one-step trace fragment η is either empty or contains a single dynamic action ι . A dynamic trace ϕ is produced by the operational semantics by concatenating

these fragments; it records the concrete allocation, read, and write events emitted by a particular execution as a list.

Trace-View Flattening

Flattening

$$\text{flatten}(\Phi) = \phi$$

$$\begin{aligned} \text{flatten}(\text{Nil}) &= [] \\ \text{flatten}(\Phi_1 \bullet \Phi_2) &= \text{flatten}(\Phi_1) \bullet \text{flatten}(\Phi_2) \end{aligned}$$

$$\begin{aligned} \text{flatten}(\iota) &= [\iota] \\ \text{flatten}(\Phi_1 \parallel \Phi_2) &= \text{flatten}(\Phi_1) \bullet \text{flatten}(\Phi_2). \end{aligned}$$

Thus Φ preserves structure, while $\text{flatten}(\Phi)$ is the list trace consumed by ordinary trace coverage and finite execution statements.

Surface-effect terms and computed effects. A surface-effect term $\hat{\Theta}$ is an expression of the source language whose result type is eff . Evaluating such a term produces a computed effect Θ . Computed effects intentionally mix abstract region actions and concrete address actions: a computed effect may say only that a computation writes some location in region r , or it may say that the computation writes the particular address $l@r$. This is the mechanism that lets a surface effect be more precise than a static region effect while still being evaluated before the summarized computation runs.

Not every static or dynamic action has the same status as a computed action. Concrete reads and writes can appear in computed effects, because a surface-effect term can evaluate an argument reference and obtain its region/location pair. Concrete allocation is present in the semantic computed-action datatype and coverage relation, but the current surface-effect syntax constructs allocation summaries abstractly at the region level. Conversely, dynamic traces contain concrete allocations, reads, and writes because they describe what an execution actually did.

The surface-effect term ‘Top’ evaluates to the distinguished computed effect $\top$. It is a sound but maximally conservative approximation: every dynamic trace is covered by $\top$, but $\top$ conflicts with every computed effect and cannot pass the pair-parallel computed-effect precheck. Thus, a surface-effect term may always fall back to ‘Top’ for soundness, but doing so loses all useful separation information. By contrast, ‘Pure’ evaluates to the empty computed-effect set and summarizes computations with no dynamic actions.

The useful slogan from the earlier presentation is $\epsilon \sqsupseteq \Theta \sqsupseteq \phi$: static effects are coarser than computed effects, and computed effects are coarser than the traces they cover. In this paper the slogan is only a guide to the reader; formally, the objects live in different syntactic categories and are related by typed bridges rather than by a single global lattice.

Typed approximation bridges. The slogan $\epsilon \sqsupseteq \Theta \sqsupseteq \phi$ abbreviates two typed approximations. The left bridge compares a static effect with the *static erasure* of a finite computed effect:

$$\begin{aligned}
[\hat{A}(r)]_\epsilon &= A(r) & [\hat{R}(r)]_\epsilon &= R(r) \\
[\hat{W}(r)]_\epsilon &= W(r) & [\widehat{\text{Alloc}}(l@r)]_\epsilon &= A(r) \\
[\widehat{\text{Read}}(l@r)]_\epsilon &= R(r) & [\widehat{\text{Write}}(l@r)]_\epsilon &= W(r).
\end{aligned}$$

$$[\{\theta_1, \dots, \theta_n\}]_\epsilon = \{[\theta_1]_\epsilon, \dots, [\theta_n]_\epsilon\}, \quad \epsilon \sqsupseteq \Theta \text{ iff } \Theta \neq \top \text{ and } [\Theta]_\epsilon \subseteq \epsilon.$$

The right bridge compares a computed effect with a machine trace. For a structured trace view, the auxiliary dynamic-action map below first forgets only the intensional view structure; equivalently, it depends on the same actions as the flattened machine trace:

$$\begin{aligned}
|\text{Nil}|_{\text{dyn}} &= \emptyset & |\iota|_{\text{dyn}} &= \{\iota\} \\
|\Phi_1 \bullet \Phi_2|_{\text{dyn}} &= |\Phi_1|_{\text{dyn}} \cup |\Phi_2|_{\text{dyn}} & |\Phi_1 \parallel \Phi_2|_{\text{dyn}} &= |\Phi_1|_{\text{dyn}} \cup |\Phi_2|_{\text{dyn}}.
\end{aligned}$$

This dynamic-action view still contains concrete locations, so it is related to computed effects by the structural approximation judgment $\phi \sqsubseteq \Theta$, not by subset inclusion. If we additionally erase dynamic actions to ordinary static actions,

$$\begin{aligned}
[\text{Alloc}(l@r)]_\epsilon &= A(r) & [\text{Read}(l@r)]_\epsilon &= R(r) \\
[\text{Write}(l@r)]_\epsilon &= W(r) & |\phi|_\epsilon &= \{[\iota]_\epsilon \mid \iota \in |\phi|_{\text{dyn}}\},
\end{aligned}$$

then subset inclusion becomes well typed at the static level:

$$\phi \sqsubseteq \Theta \quad \text{and} \quad \Theta \neq \top \quad \implies \quad |\phi|_\epsilon \subseteq [\Theta]_\epsilon.$$

Consequently, when both bridges hold for a finite computed effect, the informal three-level comparison can be read as

$$|\phi|_\epsilon \subseteq [\Theta]_\epsilon \subseteq \epsilon.$$

Computed-effect coverage. The right-hand bridge, $\Theta \sqsupseteq \phi$, is made precise by the trace-to-computed-effect judgment $\phi \sqsubseteq \Theta$. This judgment does not evaluate a trace and does not turn an execution trace into a computed effect. Instead, in the mechanization it checks whether a computed effect covers each dynamic action in the machine trace list. The same notation is lifted to structured trace views only as a convenience: $\Phi \sqsubseteq \Theta$ is the structured presentation of $\text{flatten}(\Phi) \sqsubseteq \Theta$. The action case delegates to an action-in-effect relation $\iota \preceq \Theta$.

Action Coverage			
Action-in-effect coverage			
$\iota \preceq \Theta$			
$\frac{[\text{ACT-TOP}]}{\iota \preceq \top}$	$\frac{[\text{ACT-ALLOC-ABS}]}{\widehat{A}(r) \in \vartheta \quad \text{Alloc}(l@r) \preceq \vartheta}$	$\frac{[\text{ACT-ALLOC-CONC}]}{\widehat{\text{Alloc}}(l@r) \in \vartheta \quad \text{Alloc}(l@r) \preceq \vartheta}$	$\frac{[\text{ACT-READ-CONC}]}{\widehat{\text{Read}}(l@r) \in \vartheta \quad \text{Read}(l@r) \preceq \vartheta}$
$\frac{[\text{ACT-READ-ABS}]}{\widehat{R}(r) \in \vartheta \quad \text{Read}(l@r) \preceq \vartheta}$	$\frac{[\text{ACT-WRITE-CONC}]}{\widehat{\text{Write}}(l@r) \in \vartheta \quad \text{Write}(l@r) \preceq \vartheta}$	$\frac{[\text{ACT-WRITE-ABS}]}{\widehat{W}(r) \in \vartheta \quad \text{Write}(l@r) \preceq \vartheta}$	

Trace/View Coverage	
Trace/view-to-computed-effect coverage	
$\phi \sqsubseteq \Theta \quad \Phi \sqsubseteq \Theta$	
$\frac{[\text{TRACE-NIL}]}{\text{Nil} \sqsubseteq \Theta}$	$\frac{[\text{TRACE-ELEM}]}{\iota \preceq \Theta \quad \iota \sqsubseteq \Theta}$
$\frac{[\text{TRACE-SEQ}]}{\Phi_1 \sqsubseteq \Theta \quad \Phi_2 \sqsubseteq \Theta \quad \Phi_1 \bullet \Phi_2 \sqsubseteq \Theta}$	$\frac{[\text{TRACE-PAR}]}{\Phi_1 \sqsubseteq \Theta \quad \Phi_2 \sqsubseteq \Theta \quad \Phi_1 \parallel \Phi_2 \sqsubseteq \Theta}$

For finite computed effects, the structural judgment has the expected action-set characterization:

$$\phi \sqsubseteq \Theta \quad \iff \quad \forall \iota \in |\phi|_{\text{dyn}}. \iota \preceq \Theta.$$

This equivalence explains the role of the two trace views above. The dynamic view $|\phi|_{\text{dyn}}$ keeps concrete locations and therefore compares with computed effects through $\sqsubseteq$, not through raw subset inclusion. Only after static erasure do ordinary subset comparisons become well typed: if $\phi \sqsubseteq \Theta$ and $\Theta \neq \top$, then $|\phi|_\epsilon \subseteq [\Theta]_\epsilon$.

The case $\Theta = \top$ is the conservative escape hatch for this right-hand bridge: it covers every dynamic trace, but it is not a finite computed-action set whose erasure recovers useful static precision. The mechanization defines $\top$, the empty computed effect, and union of computed effects, but it does not require a single lattice over ϵ , Θ , and ϕ .

The left-hand bridge $\epsilon \sqsupseteq \Theta$ therefore uses the erasure morphism $[\cdot]_\epsilon$ to forget the extra precision of computed effects. It erases both the abstract/concrete distinction and concrete locations: for example, $\widehat{\text{Read}}(l@r)$ and $\widehat{R}(r)$ both collapse to the static region action $R(r)$.

The right-hand bridge keeps the concrete actions in the trace and is expressed by $\phi \sqsubseteq \Theta$. This is the bridge established by surface-effect correctness: evaluating a verified surface-effect term produces a computed effect that covers the trace of the summarized computation. A later client can combine this bridge with computed-effect disjointness and non-conflict when it wants to reason about separation of two traces.

Computed-effect conflict and disjointness. The pair-parallel precheck is stated entirely over computed effects. The symbol $\boxtimes$ is the element-level conflict relation on computed actions, while $\otimes$ is the existential conflict relation lifted to computed effects: we write $\theta_1 \boxtimes \theta_2$ for conflict between computed actions and $\Theta_1 \otimes \Theta_2$ for the corresponding conflict predicate between computed effects. We therefore write $\neg(\Theta_1 \otimes \Theta_2)$ for the successful computed-effect check and $\Theta_1 \otimes \Theta_2$ for its failure. The successful check is the decidable, finite-effect version of “no computed conflict”. It succeeds only for two finite computed-effect sets and fails for ‘Top’. On finite sets, it checks that no action from one side conflicts with an action from the other. The older trace-composition lemmas are phrased with a separate universal disjointness predicate $\text{Disjoint}(\Theta_1, \Theta_2)$ and $\neg(\Theta_1 \otimes \Theta_2)$; the runtime rule packages the finite computed-effect check into the single decidable precheck.

At the trace level, $\boxtimes$ is also used for dynamic trace conflict; the types of its operands distinguish $\theta_1 \boxtimes \theta_2$ from $\phi_1 \boxtimes \phi_2$. We write $\neg(\phi_1 \boxtimes \phi_2)$ for the successful final trace check and $\phi_1 \boxtimes \phi_2$ for its failure.

Conflict Relations		
Dynamic-action conflict		
$\iota_1 \boxtimes \iota_2$		
$\frac{[\text{D-READ-WRITE}]}{\text{Read}(l@r) \boxtimes \text{Write}(l@r)}$	$\frac{[\text{D-WRITE-READ}]}{\text{Write}(l@r) \boxtimes \text{Read}(l@r)}$	$\frac{[\text{D-WRITE-WRITE}]}{\text{Write}(l@r) \boxtimes \text{Write}(l@r)}$
Dynamic-trace conflict		
$\phi_1 \boxtimes \phi_2$		
$\frac{[\text{D-TRACE}] \quad \iota_1 \in \phi_1 _{\text{dyn}} \quad \iota_2 \in \phi_2 _{\text{dyn}} \quad \iota_1 \boxtimes \iota_2}{\phi_1 \boxtimes \phi_2}$		
Computed-action conflict		
$\theta_1 \boxtimes \theta_2$		
$\frac{[\text{C-READ-WRITE}] \quad \theta = \widehat{\text{Read}}(l@r) \vee \theta = \widehat{\text{Write}}(l@r) \vee \theta = \widehat{\text{R}}(r) \vee \theta = \widehat{\text{W}}(r)}{\widehat{\text{W}}(r) \boxtimes \theta \wedge \widehat{\text{Write}}(l@r) \boxtimes \theta}$		
Computed-effect conflict		
$\Theta_1 \otimes \Theta_2$		
$\frac{[\text{C-EXT-THETA}] \quad \theta_1 \in \Theta_1 \quad \theta_2 \in \Theta_2 \quad \theta_1 \boxtimes \theta_2}{\Theta_1 \otimes \Theta_2}$	$\frac{[\text{C-TOP}]}{\Theta \otimes \top}$	

3 Syntax

The language is a region-polymorphic, call-by-value simply typed λ -calculus with mutable references and recursive functions. The distinctive syntactic form is the recursive abstraction $\mu f. \lambda x. \{e_b \mid e_s\}$, which packages two source expressions: the computational body e_b and the surface-effect body e_s . Computational application, written $\text{capp } e_f e_a$, runs e_b ; effect application, written $\text{eapp } e_f e_a$, runs e_s and must produce a value of type eff . The checked pair-parallel expression $\langle e_1, e_2 \rangle$ evaluates the surface-effect terms for its two branches before running the branch computations and returns a pair value. The checked typing and machine rules specialize this form to computational applications,

$\langle \text{capp } e_{f1} e_{a1}, \text{capp } e_{f2} e_{a2} \rangle$, so that the corresponding effect applications $\text{eapp } e_{fi} e_{ai}$ are determined by the branch syntax.

The grammar below presents the language in a compact, framed form. It uses the current terminology. A static region p is either a region variable ρ or a concrete region r . A surface-effect term is not a separate run-time language: it is source syntax of type eff . The meta-variable $\hat{\Theta}$ only isolates the expressions normally used as executable surface effects. In the mechanized syntax, Pure is represented by the empty effect expression, while Top is the conservative effect expression.

Language Syntax

$$\begin{array}{ll}
\textit{Static regions} & p ::= \rho \mid r. \\
\textit{Types} & \tau ::= \text{nat} \mid \text{bool} \mid \text{unit} \mid \text{eff} \\
& \mid \text{ref } p \tau \mid \tau_1 \times \tau_2 \mid \tau_x \rightarrow \{\tau_c ! \epsilon_c \mid \text{eff} ! \epsilon_s\} \mid \forall \rho. \tau ! \epsilon. \\
\textit{Expressions} & e ::= x \mid n \mid b \mid \mu f. \lambda x. \{e_b \mid e_s\} \mid \Lambda \rho. e \\
& \mid \text{capp } e e \mid \text{eapp } e e \mid e[p] \mid \text{cond } (e, e, e) \\
& \mid \text{alloc } p e \mid \text{get } p e \mid \text{set } p e e \\
& \mid e + e \mid e - e \mid e * e \mid e = e \\
& \mid \langle e_1, e_2 \rangle \mid \hat{\Theta}. \\
\textit{Surface-effect terms} & \hat{\Theta} ::= \text{Pure} \mid \hat{A}(p) \mid \hat{R}(p) \mid \hat{W}(p) \\
& \mid \widehat{\text{Read}}(e) \mid \widehat{\text{Write}}(e) \mid \hat{\Theta}_1 \oplus \hat{\Theta}_2 \mid \text{Top}.
\end{array}$$

Types follow the region-polymorphic simply typed calculus used throughout the formalization. The base types nat , bool , and unit classify numbers, booleans, and the result of destructive updates; eff classifies computed-effect values Θ ; and $\tau_1 \times \tau_2$ classifies pair values produced by checked pair-parallel expressions. A reference type $\text{ref } p \tau$ describes a mutable cell of contents type τ allocated in region p , where p may be a region variable or a concrete region. Function types record two latent effects: in $\tau_x \rightarrow \{\tau_c ! \epsilon_c \mid \text{eff} ! \epsilon_s\}$, ϵ_c is the static effect of the computational body and ϵ_s is the static effect of the executable surface-effect body. Region-polymorphic types $\forall \rho. \tau ! \epsilon$ abstract over regions and record the latent effect of the body under that abstraction.

Expressions include variables, numeric and Boolean constants, recursive computation/surface-effect abstractions, region abstractions, computational applications, surface-effect applications, region applications, conditionals, arithmetic and equality, heap operations, checked pair-parallel expressions, and surface-effect terms. In $\mu f. \lambda x. \{e_b \mid e_s\}$, the argument variable x and recursive variable f are available in both e_b and e_s . The type system later requires the executable surface-effect evaluation attached to a checked computation to be heap-neutral: it may inspect memory, but it may not allocate or write while computing the effect value.

Returning to Fig. 2, the type of `incr` records only the coarse region effect $\{R(\rho), W(\rho)\}$. The surface-effect application `incr_eff[7] x` is evaluated before the corresponding computation and can produce a computed effect mentioning the concrete region/location pair of `x`. The next section gives the small-step semantics that performs this pre-execution evaluation.

4 Operational Semantics

The operational semantics has one primitive execution relation: the continuation-machine step $s \longrightarrow s' ; \eta$. The other judgments below are derived or auxiliary relations built around that step. They are kept distinct because they play different roles in the metatheory.

Layer	Judgment	Role
machine step	$s \longrightarrow s'; \eta$	one continuation-machine step emitting one trace fragment
finite execution	$s \longrightarrow^* s'; \phi$	accumulated machine trace, represented as a list of actions
checked finite execution	$s \longrightarrow_{\text{chk}}^* s'; \phi$	execution variant that treats failed pair-parallel prechecks as checked failure
scheduled	$s \Longrightarrow_{\text{sch}}^{\Phi_L, \Phi_R} s'$	finite run-phase scheduler built from branch machine steps
pair-parallel run		

The notation is intentionally separated. Machine execution *produces* trace fragments; the finite-run relation concatenates them into a list trace ϕ . A one-step transition emits only a fragment η : administrative steps emit Nil, while heap operations emit singleton fragments such as $\text{Alloc}(l@r)$, $\text{Read}(l@r)$, and $\text{Write}(l@r)$. The trace of a longer execution is not stored in the ordinary state or continuation; the pair-parallel runtime state temporarily stores the two branch traces so it can perform the final checked trace-disjointness test. Checked finite execution uses the same trace fragments, but changes the observable outcome of a failed pair-parallel computed-effect precheck from sequential fallback to checked failure.

Operational Domains

$E \in \text{Env} \triangleq \text{Variables} \rightarrow \text{Values}$	variable assignment
$P \in \text{Rgn} \triangleq \text{RgnVars} \rightarrow \text{RgnConst}$	regions in scope
$H \in \text{Heap} \triangleq \text{Addr} \rightarrow \text{Values}$	heap assignment
$\langle E, P, \varphi \rangle \in \text{Closures} \triangleq \text{Env} \times \text{Rgn} \times \text{Expressions}$	function closures
$v \in \text{Values} \triangleq \text{eff}(\Theta) \mid (v_1, v_2) \mid n \mid b \mid () \mid l@r \mid \langle E, P, \varphi \rangle$	dynamic values
$s \in \text{State} \triangleq \langle H, E, P, e, k \rangle \mid \langle H, v, k \rangle$	machine states
$k \in \text{Kont} \triangleq \text{doneK} \mid K_f^\mu(e, E, P, k) \mid K_f^e(e, E, P, k)$	continuations
$\mid K_a^\mu(E, P, f, x, e, e, k) \mid K_a^e(E, P, f, x, e, e, k)$	
$\mid K^{\text{ref}}(p, P, k) \mid K^{\text{get}}(p, P, k) \mid K^{\text{set}}(p, l, P, k)$	
$\mid K^{\text{read}}(k) \mid K^{\text{write}}(k) \mid K^{\text{concat}}(\Theta, k)$	
$\mid K_1^\parallel(\vec{e}, E, P, k) \mid K_2^\parallel(\vec{e}, E, P, \Theta, k)$	
$\mid K_{F1}^\parallel(e, E, P, k) \mid K_{F2}^\parallel(v, k).$	

The initial state for expression e is the expression state with the final continuation:

$$\text{init}(H, E, P, e) \triangleq \langle H, E, P, e, \text{doneK} \rangle.$$

Finite Machine Executions

Finite execution

$$s \longrightarrow^* s'; \phi$$

$$\frac{[\text{TR-REFL}]}{s \longrightarrow^* s; \text{Nil}} \qquad \frac{[\text{TR-STEP}]}{s \longrightarrow^* s_2; \eta \bullet \phi} \quad \frac{s \longrightarrow s_1; \eta \quad s_1 \longrightarrow^* s_2; \phi}{s \longrightarrow^* s_2; \eta \bullet \phi}$$

Each finite execution accumulates one-step fragments by induction; $\bullet$ denotes list concatenation for machine traces and Nil is the empty trace. We write $\phi \equiv_{\text{perm}} \phi'$ when two list traces contain the same actions up to permutation. Ordinary expression evaluation is the special case $\text{init}(H, E, P, e) \longrightarrow^* \text{done}(H', v); \phi$.

The checked execution relation is the reflexive-transitive closure of a checked one-step relation $s \longrightarrow_{\text{chk}} s'; \eta$. Away from pair-parallel checking and pair-run states, checked steps agree with ordinary machine steps. At a failed pair-parallel computed-effect precheck, ordinary execution falls back to

sequential evaluation, while checked execution reaches the terminal checked failure state. In a pair-run state, checked branch steps use $\longrightarrow_{\text{chk}}$ recursively in the selected branch and otherwise perform the same heap rebasing and trace accumulation as the ordinary run-phase rules. The exceptional failed-precheck rule is displayed with the pair-parallel rules below.

Checked Finite Machine Executions	
Checked finite execution	$s \longrightarrow_{\text{chk}}^* s' ; \phi$
$\frac{[\text{CHK-REFL}]}{s \longrightarrow_{\text{chk}}^* s ; \text{Nil}}$	$\frac{[\text{CHK-STEP}]}{s \longrightarrow_{\text{chk}}^* s_1 ; \eta \quad s_1 \longrightarrow_{\text{chk}}^* s_2 ; \phi \quad s \longrightarrow_{\text{chk}}^* s_2 ; \eta \bullet \phi}$

The scheduled pair-run relation introduced below is finite in the same inductive sense, but it is not another reflexive-transitive closure of the ordinary global step relation. Each nonfinal scheduled rule consumes one ordinary machine step from one branch and then recurses, so the relation represents the possible interleavings of the two branch executions rather than only the ordinary left-to-right machine order.

The continuation machine accumulates list traces, not views. When a theorem must remember sequencing structure, we use the auxiliary judgment $\text{view}(s, \Phi, s')$, defined by the rules below. Its trace component is recovered by the flattening map from Section 2.

Trace-View Executions	
Structured finite execution	$\text{view}(s, \Phi, s')$
$\frac{[\text{VIEW-REFL}]}{\text{view}(s, \text{Nil}, s)}$	
$\frac{[\text{VIEW-STEP}]}{\text{view}(s, \eta, s') \quad s \longrightarrow s' ; \eta}$	$\frac{[\text{VIEW-SEQ}]}{\text{view}(s_1, \Phi_1, s_2) \quad \text{view}(s_2, \Phi_2, s_3) \quad \text{view}(s_1, \Phi_1 \bullet \Phi_2, s_3)}$

Trace-view execution is a structured presentation of finite machine execution: if $\text{view}(s, \Phi, s')$, then $s \longrightarrow^* s' ; \text{flatten}(\Phi)$; conversely, every finite machine execution has a trace view with the same flattened trace. Thus flatten appears at the boundary between view-level statements and ordinary finite executions.

Terminal states $\text{done}(H, v)$ and $\text{error}(H)$ have no outgoing machine step. The error state is the checked-failure terminal used by checked execution when a pair-parallel computed-effect precheck fails, and by the ordinary run phase when the final branch-trace check fails. Continuations are bookkeeping devices for the call-by-value evaluation order. A frame records the expression still to be evaluated, the environment and region map needed to resume evaluation, and the continuation to resume afterward. Application frames first remember the unevaluated argument while the function position is evaluated, then remember the closure while the argument is evaluated, and finally enter either the computational body or the surface-effect body with the recursive name and argument bound. Heap-operation frames remember the pending operation and the region environment while a location or value subexpression is evaluated. The trace emitted while evaluating functions, arguments, or surface-effect terms is the machine trace of that evaluation. A computed effect is the value returned by evaluating the corresponding surface-effect expression.

One machine step

$$s \longrightarrow s'; \eta$$

$$\begin{array}{c} \text{[S-CONST]} \\ \hline \langle H, E, P, \text{const } n, k \rangle \longrightarrow \langle H, n, k \rangle; \text{Nil} \end{array} \qquad \begin{array}{c} \text{[S-VAR]} \\ \hline \frac{E(x) = v}{\langle H, E, P, x, k \rangle \longrightarrow \langle H, v, k \rangle; \text{Nil}} \end{array}$$

$$\begin{array}{c} \text{[S-DONE]} \\ \hline \langle H, v, \text{doneK} \rangle \longrightarrow \text{done}(H, v); \text{Nil} \end{array}$$

$$\begin{array}{c} \text{[S-CLOS]} \\ \hline \frac{\varphi = \mu f. \lambda x. \{e_b \mid e_s\} \vee \varphi = \Lambda p. e}{\langle H, E, P, \varphi, k \rangle \longrightarrow \langle H, \langle E, P, \varphi \rangle, k \rangle; \text{Nil}} \end{array}$$

$$\begin{array}{c} \text{[S-CAPP-FUN]} \\ \hline \langle H, E, P, \text{capp } e_f e_a, k \rangle \longrightarrow \langle H, E, P, e_f, K_f^\mu(e_a, E, P, k) \rangle; \text{Nil} \end{array}$$

$$\begin{array}{c} \text{[S-CAPP-ARG]} \\ \hline \frac{v_f = \langle E', P', \mu f. \lambda x. \{e_b \mid e_s\} \rangle}{\langle H, v_f, K_f^\mu(e_a, E, P, k) \rangle \longrightarrow \langle H, E, P, e_a, K_a^\mu(E', P', f, x, e_b, e_s, k) \rangle; \text{Nil}} \end{array}$$

$$\begin{array}{c} \text{[S-CAPP-BODY]} \\ \hline \frac{E'' = E'[f \mapsto v_f][x \mapsto v_a] \quad v_f = \langle E', P', \mu f. \lambda x. \{e_b \mid e_s\} \rangle}{\langle H, v_a, K_a^\mu(E', P', f, x, e_b, e_s, k) \rangle \longrightarrow \langle H, E'', P', e_b, k \rangle; \text{Nil}} \end{array}$$

$$\begin{array}{c} \text{[S-EAPP-FUN]} \\ \hline \langle H, E, P, \text{eapp } e_f e_a, k \rangle \longrightarrow \langle H, E, P, e_f, K_f^e(e_a, E, P, k) \rangle; \text{Nil} \end{array}$$

$$\begin{array}{c} \text{[S-EAPP-ARG]} \\ \hline \frac{v_f = \langle E', P', \mu f. \lambda x. \{e_b \mid e_s\} \rangle}{\langle H, v_f, K_f^e(e_a, E, P, k) \rangle \longrightarrow \langle H, E, P, e_a, K_a^e(E', P', f, x, e_b, e_s, k) \rangle; \text{Nil}} \end{array}$$

$$\begin{array}{c} \text{[S-EAPP-BODY]} \\ \hline \frac{E'' = E'[f \mapsto v_f][x \mapsto v_a] \quad v_f = \langle E', P', \mu f. \lambda x. \{e_b \mid e_s\} \rangle}{\langle H, v_a, K_a^e(E', P', f, x, e_b, e_s, k) \rangle \longrightarrow \langle H, E'', P', e_s, k \rangle; \text{Nil}} \end{array}$$

$$\begin{array}{c} \text{[S-ALLOC]} \\ \hline \frac{P(p) = r \quad \text{fresh}(H, r) = l}{\langle H, v, K^{ref}(p, P, k) \rangle \longrightarrow \langle H[(r, l) \mapsto v], l @ r, k \rangle; \text{Alloc}(l @ r)} \end{array}$$

$$\begin{array}{c} \text{[S-GET]} \\ \hline \frac{P(p) = r \quad H(r, l) = v}{\langle H, l @ r, K^{get}(p, P, k) \rangle \longrightarrow \langle H, v, k \rangle; \text{Read}(l @ r)} \end{array}$$

$$\begin{array}{c} \text{[S-SET]} \\ \hline \frac{P(p) = r \quad H(r, l) \neq \perp}{\langle H, v, K^{set}(p, l, P, k) \rangle \longrightarrow \langle H[(r, l) \mapsto v], (), k \rangle; \text{Write}(l @ r)} \end{array}$$

Effect-valued expression steps. Surface-effect terms are ordinary expressions whose values are computed effects. Abstract surface-effect forms return region-level actions without touching the heap; concrete surface-effect forms first evaluate their argument and then return a concrete computed action. These rules are ordinary machine steps and therefore emit Nil.

Effect-Valued Expression Steps

One machine step

$$s \longrightarrow s'; \eta$$

[S-ABST-READ]

$$\frac{P(p) = r}{\langle H, E, P, \widehat{R}(p), k \rangle \longrightarrow \langle H, R(r), k \rangle; \text{Nil}}$$

[S-ABST-WRITE]

$$\frac{P(p) = r}{\langle H, E, P, \widehat{W}(p), k \rangle \longrightarrow \langle H, W(r), k \rangle; \text{Nil}}$$

[S-ABST-ALLOC]

$$\frac{P(p) = r}{\langle H, E, P, \widehat{A}(p), k \rangle \longrightarrow \langle H, A(r), k \rangle; \text{Nil}}$$

[S-CONC-READ]

$$\langle H, l@r, K^{read}(k) \rangle \longrightarrow \langle H, \widehat{\text{Read}}(l@r), k \rangle; \text{Nil}$$

[S-CONC-WRITE]

$$\langle H, l@r, K^{write}(k) \rangle \longrightarrow \langle H, \widehat{\text{Write}}(l@r), k \rangle; \text{Nil}$$

[S-CONCAT]

$$\langle H, \Theta_2, K^{concat}(\Theta_1, k) \rangle \longrightarrow \langle H, \Theta_1 \cup \Theta_2, k \rangle; \text{Nil}$$

[S-TOP]

$$\langle H, E, P, \text{Top}, k \rangle \longrightarrow \langle H, \top, k \rangle; \text{Nil}$$

[S-EMPTY]

$$\langle H, E, P, \text{Pure}, k \rangle \longrightarrow \langle H, \emptyset, k \rangle; \text{Nil}$$

The pair-parallel run phase uses the state $\text{pairRun}(s_1, s_2, \phi_1, \phi_2, k)$ to store two branch states, their accumulated branch traces, and the continuation that receives the final pair value.

The checked pair-parallel form is operationally specialized to two computational applications. The corresponding surface-effect applications are evaluated first. If the computed-effect precheck succeeds, the ordinary machine enters a pair-run state, runs the left branch to completion and then the right branch, accumulating separate branch traces and checking them before returning the pair value. If the computed-effect precheck fails, the ordinary machine falls back to sequential evaluation of the two computational branches. The checked execution relation uses the same successful run phase, but replaces this sequential fallback by terminal checked failure.

For compactness, write

$$\begin{aligned} \vec{e} &\triangleq (e_{f1}, e_{a1}, e_{f2}, e_{a2}) \\ e_i^\mu &\triangleq \text{capp } e_{fi} e_{ai} \\ e_i^e &\triangleq \text{eapp } e_{fi} e_{ai}. \end{aligned}$$

The run rules use two structural state operations. If s carries heap H_0 , then $\text{heap}(s) = H_0$. The state $s[H]$ is obtained from s by replacing only that carried heap by H .

Pair-Parallel Steps and Checked Precheck Failure

One machine step

$$s \longrightarrow s'; \eta$$

[S-PAIR-START]

$$\langle H, E, P, \langle e_1^\mu, e_2^\mu \rangle, k \rangle \longrightarrow \langle H, E, P, e_1^e, K_1^\parallel(\vec{e}, E, P, k) \rangle; \text{Nil}$$

[S-PAIR-EFF1]

$$\frac{}{\langle H, \text{eff}(\Theta_1), K_1^\parallel(\vec{e}, E, P, k) \rangle \longrightarrow \langle H, E, P, e_2^\varepsilon, K_2^\parallel(\vec{e}, E, P, \Theta_1, k) \rangle; \text{Nil}}$$

[S-PAIR-CHECK-OK]

$$\frac{\neg(\Theta_1 \otimes \Theta_2)}{\langle H, \text{eff}(\Theta_2), K_2^\parallel(\vec{e}, E, P, \Theta_1, k) \rangle \longrightarrow \text{pairRun}(\text{init}(H, E, P, e_1^\mu), \text{init}(H, E, P, e_2^\mu), [], [], k); \text{Nil}}$$

[S-PAIR-CHECK-FAIL]

$$\frac{\Theta_1 \otimes \Theta_2}{\langle H, \text{eff}(\Theta_2), K_2^\parallel(\vec{e}, E, P, \Theta_1, k) \rangle \longrightarrow \langle H, E, P, e_1^\mu, K_{F1}^\parallel(e_2^\mu, E, P, k) \rangle; \text{Nil}}$$

[S-PAIR-FALLBACK-L]

$$\frac{}{\langle H, v_1, K_{F1}^\parallel(e_2^\mu, E, P, k) \rangle \longrightarrow \langle H, E, P, e_2^\mu, K_{F2}^\parallel(v_1, k) \rangle; \text{Nil}}$$

[S-PAIR-FALLBACK-R]

$$\frac{}{\langle H, v_2, K_{F2}^\parallel(v_1, k) \rangle \longrightarrow \langle H, (v_1, v_2), k \rangle; \text{Nil}}$$

[C-PAIR-CHECK-FAIL]

$$\frac{\Theta_1 \otimes \Theta_2}{\langle H, \text{eff}(\Theta_2), K_2^\parallel(\vec{e}, E, P, \Theta_1, k) \rangle \longrightarrow_{\text{chk}} \text{error}(H); \text{Nil}}$$

[S-PAIR-RUN-L]

$$\frac{s_1 \longrightarrow s'_1; \eta}{\text{pairRun}(s_1, s_2, \phi_1, \phi_2, k) \longrightarrow \text{pairRun}(s'_1, s_2[\text{heap}(s'_1)], \phi_1 \bullet \eta, \phi_2, k); \eta}$$

[S-PAIR-RUN-R]

$$\frac{s_2 \longrightarrow s'_2; \eta}{\text{pairRun}(\text{done}(H_1, v_1), s_2, \phi_1, \phi_2, k) \longrightarrow \text{pairRun}(\text{done}(\text{heap}(s'_2), v_1), s'_2, \phi_1, \phi_2 \bullet \eta, k); \eta}$$

[S-PAIR-DONE-OK]

$$\frac{\neg(\phi_1 \boxtimes \phi_2)}{\text{pairRun}(\text{done}(H, v_1), \text{done}(H, v_2), \phi_1, \phi_2, k) \longrightarrow \langle H, (v_1, v_2), k \rangle; \text{Nil}}$$

[S-PAIR-DONE-FAIL]

$$\frac{\phi_1 \boxtimes \phi_2}{\text{pairRun}(\text{done}(H, v_1), \text{done}(H, v_2), \phi_1, \phi_2, k) \longrightarrow \text{error}(H); \text{Nil}}$$

The ordinary semantics uses [S-Pair-Check-Fail] for a failed computed-effect precheck. The checked relation uses [C-Pair-Check-Fail] in its place. Its pair-run left and right rules are the checked counterparts of [S-Pair-Run-L] and [S-Pair-Run-R], with $\longrightarrow_{\text{chk}}$ in the branch premise; the state update, heap rebasing, and emitted trace fragment are unchanged. The final pair-run cases are shared by the ordinary and checked relations. If either branch reaches $\text{error}(H)$, the pair-parallel state steps to the same checked-failure terminal state.

For the scheduler theorem, the same pair-run states are equipped with a derived scheduled relation. It does not add a new source step; it allows either live branch to take the next run-phase step while recording the two branch trace views separately. The final scheduled cases coincide with the checked pair-parallel run phase. For example, [Sch-Done-OK] has the same state-level premise and conclusion as [S-Pair-Done-OK], but it is a base case of the finite scheduled relation rather than one primitive machine step. Its branch views are Nil, Nil because the join itself emits no branch action; the accumulated branch traces ϕ_1, ϕ_2 are already stored in the `pairRun` state.

Scheduled Pair-Parallel Runs

Scheduled run

$$s \xRightarrow{\Phi_L, \Phi_R}_{\text{sch}} s'$$

[SCH-ERR-L]

$$\text{pairRun}(\text{error}(H), s_2, \phi_1, \phi_2, k) \xRightarrow{\text{Nil}, \text{Nil}}_{\text{sch}} \text{error}(H)$$

[SCH-ERR-R]

$$\text{pairRun}(s_1, \text{error}(H), \phi_1, \phi_2, k) \xRightarrow{\text{Nil}, \text{Nil}}_{\text{sch}} \text{error}(H)$$

[SCH-DONE-OK]

$$\frac{\neg(\phi_1 \boxtimes \phi_2)}{\text{pairRun}(\text{done}(H, v_1), \text{done}(H, v_2), \phi_1, \phi_2, k) \xRightarrow{\text{Nil}, \text{Nil}}_{\text{sch}} \langle H, (v_1, v_2), k \rangle}$$

[SCH-DONE-FAIL]

$$\frac{\phi_1 \boxtimes \phi_2}{\text{pairRun}(\text{done}(H, v_1), \text{done}(H, v_2), \phi_1, \phi_2, k) \xRightarrow{\text{Nil}, \text{Nil}}_{\text{sch}} \text{error}(H)}$$

[SCH-RUN-L]

$$\frac{s_1 \longrightarrow s'_1; \eta \quad \neg \exists H_e. s_2 = \text{error}(H_e) \quad \text{pairRun}(s'_1, s_2[\text{heap}(s'_1)], \phi_1 \bullet \eta, \phi_2, k) \xRightarrow{\Phi_L, \Phi_R}_{\text{sch}} s}{\text{pairRun}(s_1, s_2, \phi_1, \phi_2, k) \xRightarrow{\eta \bullet \Phi_L, \Phi_R}_{\text{sch}} s}$$

[SCH-RUN-R]

$$\frac{s_2 \longrightarrow s'_2; \eta \quad \neg \exists H_e. s_1 = \text{error}(H_e) \quad \text{pairRun}(s_1[\text{heap}(s'_2)], s'_2, \phi_1, \phi_2 \bullet \eta, k) \xRightarrow{\Phi_L, \Phi_R}_{\text{sch}} s}{\text{pairRun}(s_1, s_2, \phi_1, \phi_2, k) \xRightarrow{\Phi_L, \eta \bullet \Phi_R}_{\text{sch}} s}$$

Thus a left-branch step is the same state update as [S-Pair-Run-L]. A right-branch step may occur before the left branch has finished; after such a step the left state is rebased to the right branch's new heap. The side conditions prevent either branch from continuing to schedule steps after the other branch has already reached `error(H)`. From empty branch accumulators, the final stored branch traces are exactly `flatten( $\Phi_L$ )` and `flatten( $\Phi_R$ )`. The normalized scheduler trace view is $\Phi_L \parallel \Phi_R$, whose flattening is the canonical branch projection `flatten( $\Phi_L$ ) • flatten( $\Phi_R$ )` even when the scheduled execution interleaves the one-step fragments.

The connection back to finite machine executions is an embedding, not an identification. Let

$$s_0 = \text{pairRun}(\text{init}(H, E, P, e_L^\mu), \text{init}(H, E, P, e_R^\mu), [], [], \text{doneK}).$$

If the ordinary left-to-right machine run terminates,

$$s_0 \longrightarrow^* \text{done}(H', v); \phi,$$

then there exist branch views Φ_L, Φ_R such that

$$s_0 \xRightarrow{\Phi_L, \Phi_R}_{\text{sch}} \langle H', v, \text{doneK} \rangle \quad \text{and} \quad \phi \equiv_{\text{perm}} \text{flatten}(\Phi_L \parallel \Phi_R).$$

The converse is intentionally not stated as an ordinary finite execution: arbitrary scheduled runs may take a right-branch step before the ordinary machine would do so.

Surface-effect evaluation is proved heap-neutral from typing: if a well-typed surface-effect execution emits a heap-neutral trace, the final heap is the initial heap. A surface effect may *denote* an

allocation through an abstract allocation action without performing an allocation while the surface-effect term itself is evaluated.

5 Static Semantics

The typing rules for the ordinary surface-effect language are given in the present section. A variable context Γ is a total map that records free variables and their types and a region context Δ is a set of region variables in scope. The judgment $\Gamma; \Delta \vdash e : \tau ! \epsilon$ asserts that expression e has type τ under Γ and Δ and the latent (static) effect ϵ . The rules below display the checked typing layer used by the proofs: besides ordinary typing and well-formedness, the abstraction and region-abstraction cases carry the surface-effect correctness obligations. The correctness relation between a computation and its surface-effect expression is denoted by the binary relation ($\blacktriangleleft$). The rule [T-CAPP] declares the return type of a computational application, **capp**, and the same reasoning is made for **eapp**, whose return type is always **eff**.

Because the mechanization uses logical equality when proving soundness properties about surface-effect terms, types are represented in a locally nameless style (Charguéraud, 2012), so that quantified types are equivalent up to α -conversion. Bound variables are represented with de Bruijn indices and free-variable names are left unchanged. This representation lets substitution be stated directly by type-level β -reduction.

Static Semantics	
Well-typed expressions	$\Gamma; \Delta \vdash e : \tau ! \epsilon$
[T-VAR]	
$\frac{\Gamma(x) = \tau}{\Gamma; \Delta \vdash x : \tau ! \emptyset}$	
[T-CONST]	
$\frac{}{\Gamma; \Delta \vdash n : \text{nat} ! \emptyset}$	
[T-BOOL]	
$\frac{}{\Gamma; \Delta \vdash b : \text{bool} ! \emptyset}$	
[T-ABST]	
$\frac{\Gamma, f : \tau_f, x : \tau_x; \Delta \vdash e_b : \tau_b ! \epsilon_b \quad \tau_f = \tau_x \rightarrow \{\tau_b ! \epsilon_b \mid \text{eff} ! \epsilon_s\} \quad \Gamma, f : \tau_f, x : \tau_x; \Delta \vdash e_s : \text{eff} ! \epsilon_s \quad \Gamma, f : \tau_f, x : \tau_x; \Delta; P \vdash e_b \blacktriangleleft e_s}{\Gamma; \Delta \vdash \mu f. \lambda x. \{e_b \mid e_s\} : \tau_f ! \emptyset}$	
[T-CAPP]	
$\frac{\Gamma; \Delta \vdash e_f : \tau_x \rightarrow \{\tau_b ! \epsilon_b \mid \text{eff} ! \epsilon_s\} ! \emptyset \quad \Gamma; \Delta \vdash e_a : \tau_x ! \epsilon_a}{\Gamma; \Delta \vdash \text{capp } e_f e_a : \tau_b ! \epsilon_b \cup \epsilon_a}$	
[T-EAPP]	
$\frac{\Gamma; \Delta \vdash e_f : \tau_x \rightarrow \{\tau_b ! \epsilon_b \mid \text{eff} ! \epsilon_s\} ! \emptyset \quad \Gamma; \Delta \vdash e_a : \tau_x ! \epsilon_a}{\Gamma; \Delta \vdash \text{eapp } e_f e_a : \text{eff} ! \epsilon_s \cup \epsilon_a}$	
[T-RGN-ABST]	
$\frac{\Gamma; \Delta, \rho \vdash e : \tau ! \epsilon \quad \rho \notin \Delta \quad \Gamma; \Delta, \rho; P \vdash e \blacktriangleleft \text{Pure}}{\Gamma; \Delta \vdash \Lambda \rho. e : (\forall \rho. \tau ! \epsilon) ! \emptyset}$	
[T-RGN-APP]	
$\frac{\Gamma; \Delta \vdash e : \forall \rho. \tau ! \epsilon \quad \Delta \vdash p}{\Gamma; \Delta \vdash e[p] : \tau[p/\rho] ! \epsilon[p/\rho]}$	
[T-PAIRPAR]	
$\frac{\Gamma; \Delta \vdash \text{capp } e_{f1} e_{a1} : \tau_1 ! \epsilon_1 \quad \Gamma; \Delta \vdash \text{eapp } e_{f1} e_{a1} : \text{eff} ! \epsilon_{s1} \quad \Gamma; \Delta \vdash \text{eapp } e_{f2} e_{a2} : \text{eff} ! \epsilon_{s2}}{\Gamma; \Delta \vdash (\text{capp } e_{f1} e_{a1}, \text{capp } e_{f2} e_{a2}) : \tau_1 \times \tau_2 ! \epsilon_1 \cup \epsilon_{s1} \cup \epsilon_2 \cup \epsilon_{s2}}$	

$\frac{[\text{T-COND}]}{\Gamma; \Delta \vdash e_c : \text{bool}! \epsilon_c \quad \Gamma; \Delta \vdash e_t : \tau! \epsilon_t \quad \Gamma; \Delta \vdash e_f : \tau! \epsilon_f} \quad \Gamma; \Delta \vdash \text{cond}(e_c, e_t, e_f) : \tau! \epsilon_c \cup \epsilon_t \cup \epsilon_f$		$\frac{[\text{T-ARITH}]}{\Gamma; \Delta \vdash e_1 : \tau_1! \epsilon_1 \quad \Gamma; \Delta \vdash e_2 : \tau_2! \epsilon_2} \quad \Gamma; \Delta \vdash e_1 \odot e_2 : \tau! \epsilon_1 \cup \epsilon_2$	
$\frac{[\text{T-ALLOC}]}{\Gamma; \Delta \vdash e : \tau! \epsilon \quad \Delta \vdash p} \quad \Gamma; \Delta \vdash \text{alloc } p e : \text{ref } p \tau! \epsilon \cup \{\mathbf{A}(p)\}$		$\frac{[\text{T-GET}]}{\Gamma; \Delta \vdash e : \text{ref } p \tau! \epsilon} \quad \Gamma; \Delta \vdash \text{get } p e : \tau! \epsilon \cup \{\mathbf{R}(p)\}$	
$\frac{[\text{T-SET}]}{\Gamma; \Delta \vdash e_r : \text{ref } p \tau! \epsilon_r \quad \Gamma; \Delta \vdash e_v : \tau! \epsilon_v} \quad \Gamma; \Delta \vdash \text{set } p e_r e_v : \text{unit}! \epsilon_r \cup \epsilon_v \cup \{\mathbf{W}(p)\}$			
$\frac{[\text{T-ABST-ALLOC}]}{\Gamma; \Delta \vdash \widehat{\mathbf{A}}(p) : \text{eff}! \emptyset}$		$\frac{[\text{T-ABST-READ}]}{\Gamma; \Delta \vdash \widehat{\mathbf{R}}(p) : \text{eff}! \emptyset}$	
$\frac{[\text{T-ABST-WRITE}]}{\Gamma; \Delta \vdash \widehat{\mathbf{W}}(p) : \text{eff}! \emptyset}$			
$\frac{[\text{T-CONC-READ}]}{\Gamma; \Delta \vdash e : \text{ref } p \tau! \epsilon} \quad \Gamma; \Delta \vdash \widehat{\text{Read}}(e) : \text{eff}! \epsilon$		$\frac{[\text{T-CONC-WRITE}]}{\Gamma; \Delta \vdash e : \text{ref } p \tau! \epsilon} \quad \Gamma; \Delta \vdash \widehat{\text{Write}}(e) : \text{eff}! \epsilon$	
$\frac{[\text{T-CONCAT}]}{\Gamma; \Delta \vdash \widehat{\Theta}_1 : \text{eff}! \epsilon_1 \quad \Gamma; \Delta \vdash \widehat{\Theta}_2 : \text{eff}! \epsilon_2} \quad \Gamma; \Delta \vdash \widehat{\Theta}_1 \oplus \widehat{\Theta}_2 : \text{eff}! \epsilon_1 \cup \epsilon_2$		$\frac{[\text{T-TOP}]}{\Gamma; \Delta \vdash \text{Top} : \text{eff}! \emptyset}$	
$\frac{[\text{T-PURE}]}{\Gamma; \Delta \vdash \text{Pure} : \text{eff}! \emptyset}$			

Here $\odot \in \{+, -, \times, =\}$ abbreviates the arithmetic and equality cases. The correctness premises in [T-Abst] and [T-Rgn-Abst] are read under the current region map P , matching the P -indexed back-triangle judgment below. In the mechanization, this displayed judgment corresponds to the checked typing predicate: it packages ordinary typing, regularity of contexts, and the shape obligations shown in the rules.

On static effects, write $\text{NoAlloc}(\epsilon)$ when ϵ contains no allocation action $A(p)$. The write-freedom side condition rules out writes, while NoAlloc rules out allocation. Their conjunction is the derived predicate HeapNeutral : a heap-neutral static effect has neither allocation nor write actions. Ordinary static-effect soundness is an auxiliary theorem about evaluation traces: if the static erasure $|\phi|_\epsilon$ of a trace is contained in a write-free static effect, then the trace contains no writes; if it is contained in a heap-neutral static effect, then the trace has no allocations and no writes. This is separate from the computed-effect precision bridge $\phi \sqsubseteq \Theta$, which deliberately keeps concrete locations until they are erased for static comparison. Consequently, heap-neutral surface-effect evaluation preserves the heap. This is a semantic property, not a cost claim: the development does not prove that a heap-neutral surface effect terminates or that evaluating it has constant overhead.

Corollary 1 (Empty static trace erasure implies the empty trace). *For all traces ϕ , if $|\phi|_\epsilon$ is contained in the empty static effect, then $\phi = \text{Nil}$.*

Lemma 1 (Heap-neutral traces imply heap preservation). *Given that $\text{HeapNeutral}(\phi)$, a terminal execution*

$$\text{init}(H, E, P, e) \longrightarrow^* \text{done}(H', v); \phi$$

implies that $H \equiv H'$ up to key-value pairs.

Lemma 2 (Heap-neutral static effects imply heap-neutral traces). *If $|\phi|_\epsilon \subseteq \epsilon$ and $\text{HeapNeutral}(\epsilon)$ holds for static effects, then $\text{HeapNeutral}(\phi)$ holds for the dynamic trace.*

Machine safety and static effects. Write $\text{CheckedCtx}_{\Gamma, \Delta}(H, E, P)$ for the checked run-time context used in the mechanization: the heap, environment, and region map are well formed and store resolved for Γ and Δ . Write $\text{CheckedInit}_{\Gamma, \Delta}(H, E, P, e)$ when, in addition, e is checked by the typing rules above. Write $\Gamma; \Delta \vdash_{\text{wt}} s$ for the machine-state typing invariant used by progress, and $\Sigma \vdash_{\text{st}} s : \tau$ for the store-resolved state-shape invariant used by preservation. The following three statements are the continuation-machine safety facts used by the correctness theorem.

Theorem 1 (Progress). *If $\Gamma; \Delta \vdash_{\text{wt}} s$, then either $s = \text{done}(H, v)$, $s = \text{error}(H)$, or there exist s' and η such that*

$$s \longrightarrow s'; \eta.$$

In particular, $\text{CheckedInit}_{\Gamma, \Delta}(H, E, P, e)$ implies this not-stuck property for $\text{init}(H, E, P, e)$.

Theorem 2 (Preservation). *If*

$$\Sigma \vdash_{\text{st}} s : \tau \quad \text{and} \quad s \longrightarrow s'; \eta,$$

then there exists a store typing Σ' such that

$$\Sigma' \vdash_{\text{st}} s' : \tau \quad \text{and} \quad \text{Transport}(\Sigma, s, \Sigma', s').$$

The transport relation records heap/store stability for ordinary steps and the fresh store extension for allocation steps.

Theorem 3 (Static-effect trace soundness). *Assume $\text{CheckedCtx}_{\Gamma, \Delta}(H, E, P)$. If e is checked by the typing rules above,*

$$\Gamma; \Delta \vdash e : \tau! \epsilon, \quad \text{init}(H, E, P, e) \longrightarrow^* \text{done}(H', v); \phi,$$

then the static erasure of the emitted trace is included in the resolved static effect:

$$|\phi|_{\epsilon} \subseteq \epsilon[P].$$

6 Correctness of Surface Effects

The previous section defined the typed bridge $\Theta \sqsupseteq \phi$ between computed effects and dynamic traces. This section explains how that bridge is obtained from source-level verification. The judgment

$$\Gamma; \Delta; P \vdash e \blacktriangleleft \hat{\Theta}$$

relates a computation e to a surface-effect term $\hat{\Theta}$. If e executes with trace ϕ and $\hat{\Theta}$ evaluates to a computed effect Θ , then surface-effect correctness requires

$$\Theta \sqsupseteq \phi,$$

equivalently $\phi \sqsubseteq \Theta$. The rules defining this coverage relation are part of the general effect vocabulary in Section 2; this section proves that verified surface-effect terms establish that relation.

The judgment is intentionally syntax directed but incomplete: there are infinitely many sound surface effects for a given expression, and some expressions can always be over-summarized by ‘Top’. The theorem below therefore states a soundness property for derivable summaries, not a completeness result for all possible surface effects.

The correctness theorem below follows the clean direction of the design. A surface-effect term evaluates to a computed effect, and the computation emits a trace. The theorem proves that the already-computed effect covers the computation trace:

$$\hat{\Theta} \longrightarrow^* \Theta \quad \text{and} \quad \phi \sqsubseteq \Theta.$$

The proof keeps traces and surface effects as separate objects. If evaluating a function position, an argument position, or another subexpression may perform memory actions, those actions must be accounted for by the surface-effect term and by the source-level correctness judgment itself; they are not recovered after the fact.

The back-triangle judgment says that the source term $\hat{\Theta}$ is an executable surface effect for the computation e under the current region map P . The judgment is indexed by P : for a static effect ϵ , write $\epsilon[P]$ for the effect obtained by substituting the region values in P . The following rules give the paper-facing inductive presentation. In particular, pure constants, booleans, variables, abstractions, and region abstractions are summarized by Pure, rather than by an arbitrary surface effect.

Back-Triangle Rules

Executable surface-effect correctness

$$\boxed{\Gamma; \Delta; P \vdash e \triangleleft \hat{\Theta}}$$

$$\frac{[\text{EFF-CONST}] \quad \Gamma; \Delta \vdash \text{const } n : \text{nat}! \emptyset}{\Gamma; \Delta; P \vdash \text{const } n \triangleleft \text{Pure}}$$

$$\frac{[\text{EFF-BOOL}] \quad \Gamma; \Delta \vdash \text{bool } b : \text{bool}! \emptyset}{\Gamma; \Delta; P \vdash \text{bool } b \triangleleft \text{Pure}}$$

$$\frac{[\text{EFF-VAR}] \quad \Gamma; \Delta \vdash \text{var } x : \tau! \emptyset}{\Gamma; \Delta; P \vdash \text{var } x \triangleleft \text{Pure}}$$

$$\frac{[\text{EFF-ABST}] \quad \Gamma; \Delta \vdash \mu f. \lambda x. \{e_b \mid e_e\}}{\Gamma; \Delta; P \vdash \mu f. \lambda x. \{e_b \mid e_e\} \triangleleft \text{Pure}}$$

$$\frac{[\text{EFF-RGN-ABST}]}{\Gamma; \Delta; P \vdash \lambda \rho. e \triangleleft \text{Pure}}$$

$$\frac{[\text{EFF-APP}] \quad \begin{array}{c} \Gamma; \Delta \vdash \text{capp } e_f e_a : \tau_b! \epsilon_b \quad \Gamma; \Delta \vdash \text{eapp } e_f e_a : \text{eff}! \epsilon_e \\ \text{HeapNeutral}(\epsilon_e[P]) \quad \Gamma; \Delta \vdash e_f : \tau_f! \epsilon_f \quad \Gamma; \Delta \vdash e_a : \tau_a! \epsilon_a \quad \Gamma; \Delta; P \vdash e_f \triangleleft (\text{eapp } e_f e_a) \\ \Gamma; \Delta; P \vdash e_a \triangleleft (\text{eapp } e_f e_a) \quad \text{HeapNeutral}(\epsilon_f[P]) \quad \text{HeapNeutral}(\epsilon_a[P]) \end{array}}{\Gamma; \Delta; P \vdash \text{capp } e_f e_a \triangleleft (\text{eapp } e_f e_a)}$$

$$\frac{[\text{EFF-PAIRPAR}] \quad \begin{array}{c} \Gamma; \Delta \vdash \text{capp } e_{f1} e_{a1} : \tau_1! \epsilon_1 \quad \Gamma; \Delta \vdash \text{capp } e_{f2} e_{a2} : \tau_2! \epsilon_2 \quad \text{NoAlloc}(\epsilon_1[P]) \\ \text{NoAlloc}(\epsilon_2[P]) \quad \Gamma; \Delta \vdash \text{eapp } e_{f1} e_{a1} : \text{eff}! \epsilon_{s1} \quad \Gamma; \Delta \vdash \text{eapp } e_{f2} e_{a2} : \text{eff}! \epsilon_{s2} \quad \text{HeapNeutral}(\epsilon_{s1}[P]) \\ \text{HeapNeutral}(\epsilon_{s2}[P]) \quad \Gamma; \Delta; P \vdash \text{capp } e_{f1} e_{a1} \triangleleft \text{eapp } e_{f1} e_{a1} \quad \Gamma; \Delta; P \vdash \text{capp } e_{f2} e_{a2} \triangleleft \text{eapp } e_{f2} e_{a2} \end{array}}{\Gamma; \Delta; P \vdash \langle \text{capp } e_{f1} e_{a1}, \text{capp } e_{f2} e_{a2} \rangle \triangleleft (\text{eapp } e_{f1} e_{a1}) \oplus (\text{eapp } e_{f2} e_{a2})}$$

$$\frac{[\text{EFF-RGN-APP}] \quad \begin{array}{c} \Gamma; \Delta \vdash e_r : \tau! \epsilon \quad \Gamma; \Delta \vdash \text{Pure} : \text{eff}! \emptyset \quad \Gamma; \Delta; P \vdash e_r \triangleleft \text{Pure} \end{array}}{\Gamma; \Delta; P \vdash e_r[r] \triangleleft \text{Pure}}$$

$$\frac{[\text{EFF-COND}] \quad \begin{array}{c} \Gamma; \Delta \vdash e_c : \tau_c! \epsilon_c \quad \Gamma; \Delta \vdash e_t : \tau_t! \epsilon_t \quad \Gamma; \Delta \vdash e_f : \tau_f! \epsilon_f \\ \text{HeapNeutral}(\epsilon_c[P]) \quad \Gamma; \Delta; P \vdash e_c \triangleleft \text{Pure} \quad \Gamma; \Delta; P \vdash e_t \triangleleft e_1 \quad \Gamma; \Delta; P \vdash e_f \triangleleft e_2 \end{array}}{\Gamma; \Delta; P \vdash \text{cond } e_c e_t e_f \triangleleft \text{cond } e_c e_1 e_2}$$

$$\frac{[\text{EFF-ALLOC}] \quad \begin{array}{c} \Gamma; \Delta \vdash e : \tau! \epsilon \quad \Gamma; \Delta; P \vdash e \triangleleft e_1 \end{array}}{\Gamma; \Delta; P \vdash \text{alloc } r e \triangleleft e_1 \oplus \hat{A}(r)}$$

$$\frac{[\text{EFF-GET-ABS}] \quad \begin{array}{c} \Gamma; \Delta \vdash e : \tau! \epsilon \quad \Gamma; \Delta; P \vdash e \triangleleft e_1 \end{array}}{\Gamma; \Delta; P \vdash \text{get } p e \triangleleft e_1 \oplus \hat{R}(p)}$$

$$\frac{[\text{EFF-GET-CONC}] \quad \begin{array}{c} \Gamma; \Delta \vdash e : \tau! \epsilon \quad \Gamma; \Delta; P \vdash e \triangleleft \text{Pure} \end{array}}{\Gamma; \Delta; P \vdash \text{get } r e \triangleleft \widehat{\text{Read}}(e)}$$

$$\frac{[\text{EFF-SET-ABS}] \quad \begin{array}{c} \Gamma; \Delta; P \vdash e_r \triangleleft e_1 \quad \Gamma; \Delta; P \vdash e_v \triangleleft e_2 \quad \Gamma; \Delta \vdash e_r : \tau_r! \epsilon_r \quad \text{HeapNeutral}(\epsilon_r[P]) \end{array}}{\Gamma; \Delta; P \vdash \text{set } p e_r e_v \triangleleft e_1 \oplus (e_2 \oplus \hat{W}(p))}$$

$$\begin{array}{c}
\text{[EFF-SET-CONC]} \\
\frac{\Gamma; \Delta; P \vdash e_r \blacktriangleleft e_1 \quad \Gamma; \Delta; P \vdash e_v \blacktriangleleft e_2 \quad \Gamma; \Delta \vdash e_r : \tau_r ! \epsilon_r \quad \text{HeapNeutral}(\epsilon_r[P])}{\Gamma; \Delta; P \vdash \text{set } r \ e_r \ e_v \blacktriangleleft e_1 \oplus (e_2 \oplus \widehat{\text{Write}}(e_r))} \\
\\
\text{[EFF-ARITH]} \\
\frac{\Gamma; \Delta \vdash e_1 : \tau_1 ! \epsilon_1 \quad \Gamma; \Delta; P \vdash e_1 \blacktriangleleft \widehat{\Theta}_1 \quad \Gamma; \Delta; P \vdash e_2 \blacktriangleleft \widehat{\Theta}_2 \quad \text{HeapNeutral}(\epsilon_1[P])}{\Gamma; \Delta; P \vdash e_1 \odot e_2 \blacktriangleleft \widehat{\Theta}_1 \oplus \widehat{\Theta}_2} \\
\\
\text{[EFF-TOP]} \\
\frac{}{\Gamma; \Delta; P \vdash e \blacktriangleleft \text{Top}}
\end{array}$$

Here $\odot \in \{+, -, \times, =\}$ abbreviates the four corresponding arithmetic and equality cases. The concrete read and write rules above use a concrete region value r .

A verified surface-effect term evaluates to a computed effect that covers the entire trace of the summarized computation. The checked finite execution relation $s \longrightarrow_{\text{chk}}^* s'$; ϕ is the ordinary finite machine execution relation with failed pair-parallel computed-effect prechecks reported as terminal checked failures rather than sequential fallback.

Theorem 4 (Correctness of surface-effect terms). *Assume $\text{CheckedCtx}_{\Gamma, \Delta}(H, E, P)$. If the computation and surface-effect term are related by $\Gamma; \Delta; P \vdash e_b \blacktriangleleft e_e$, and*

$$\text{init}(H, E, P, e_b) \longrightarrow_{\text{chk}}^* \text{done}(H_b, v_b); \phi_b \quad \text{and} \quad \text{init}(H, E, P, e_e) \longrightarrow^* \text{done}(H_e, \text{eff}(\Theta_e)); \phi_e,$$

then

$$\Theta_e \sqsupseteq \phi_b \quad \text{equivalently} \quad \phi_b \sqsubseteq \Theta_e.$$

The conclusion of the correctness theorem is exactly the right-hand typed bridge. In particular, it does *not* conclude that the computation trace ϕ_b is heap-neutral, nor does it prove the left-hand bridge $\epsilon \sqsupseteq \Theta_e$. The later small-step discussion reuses this approximation story for terminal determinism and terminal correctness directly over the continuation machine.

Corollary 2 (Heap-neutral, heap-preserving surface-effect evaluation). *For a well-typed computation together with its effect application, static effect soundness and the typing premises of the correctness relation imply $\text{HeapNeutral}(\phi_e)$. Therefore surface-effect evaluation preserves the initial heap: $H \equiv H_e$.*

The proof separates two facts: the computation trace is shown to satisfy $\phi_b \sqsubseteq \Theta_e$, while the trace produced by evaluating the effect application is separately shown to be heap-neutral. Thus a computed action such as $A(r)$ may *describe* a future allocation without the evaluation of the surface-effect term itself allocating memory.

Trace/surface-effect separation. The previous theorem is the only coverage principle needed for surface-effect correctness: a verified surface-effect term must cover the whole trace of its summarized computation. The dynamic-action views $|\phi|_{\text{dyn}}$ and $|\phi|_{\epsilon}$ are used only to state erasure and coverage facts about already produced traces.

If two traces are covered by disjoint and nonconflicting computed effects, then their parallel trace composition is deterministic.

Theorem 5 (Trace-composition soundness). *Let $\phi_1 \sqsubseteq \Theta_1$ and $\phi_2 \sqsubseteq \Theta_2$. If*

$$\text{Disjoint}(\Theta_1, \Theta_2) \quad \text{and} \quad \neg(\Theta_1 \otimes \Theta_2),$$

and $\text{Det}(\phi_1)$ and $\text{Det}(\phi_2)$, then

$$\text{Det}(\phi_1 \parallel \phi_2).$$

The proof first shows that a dynamic read–write or write–write conflict would induce a corresponding computed conflict, contradicting the computed-effect premise. It separately transports computed-action disjointness to every cross-trace pair of dynamic actions. The result is the deterministic parallel trace judgment above.

Combining the preceding results gives the trace-level proof dependency:

$$\begin{array}{c}
\textbf{Per-branch correctness} \\
\begin{array}{cc}
\Gamma; \Delta; P \vdash e_1 \blacktriangleleft \hat{\Theta}_1 & \Gamma; \Delta; P \vdash e_2 \blacktriangleleft \hat{\Theta}_2 \\
H_a; e_1 \rightsquigarrow \phi_1 & H_b; e_2 \rightsquigarrow \phi_2 \\
\hat{\Theta}_1 \longrightarrow^* \Theta_1 & \hat{\Theta}_2 \longrightarrow^* \Theta_2 \\
\phi_1 \sqsubseteq \Theta_1 & \phi_2 \sqsubseteq \Theta_2
\end{array} \\
\Downarrow \\
\textbf{Pair-parallel check} \\
\text{Disjoint}(\Theta_1, \Theta_2) \text{ and } \neg(\Theta_1 \otimes \Theta_2) \text{ and } \text{Det}(\phi_1) \text{ and } \text{Det}(\phi_2) \\
\Downarrow \\
\textbf{Trace composition} \\
\text{Det}(\phi_1 \parallel \phi_2) \\
\Downarrow \\
\textbf{Terminating replay consequence} \\
H_a \equiv H_b \\
(\phi_1 \parallel \phi_2, H_a) \Rightarrow^* (\text{Nil}, H_1) \quad \wedge \quad (\phi_1 \parallel \phi_2, H_b) \Rightarrow^* (\text{Nil}, H_2) \\
\implies H_1 \equiv H_2
\end{array}$$

Here $H; e \rightsquigarrow \phi$ refines the trace-only abbreviation used in Figure 1 by naming the initial heap, while still suppressing the environment, region map, final value, and continuation. The judgment $(\phi, H) \Rightarrow^* (\text{Nil}, H')$ denotes terminating replay of trace ϕ from heap H to final heap H' , and $\hat{\Theta}_i \longrightarrow^* \Theta_i$ abbreviates terminating evaluation of the surface effect to a computed effect.

Runtime Definitions

$\Sigma \in \text{StoreTy} \triangleq \text{Addr} \rightarrow \text{Type}$	heap type assignment
$H_1 \equiv H_2 \triangleq \forall a. H_1(a) = H_2(a)$	heap lookup equivalence
$\text{CheckedCtx}_{\Gamma, \Delta}(H, E, P)$	checked run-time context
$\text{CheckedInit}_{\Gamma, \Delta}(H, E, P, e)$	checked initial expression context
$\text{NotStuck}(s) \triangleq s = \text{done}(H, v) \vee s = \text{error}(H)$ $\vee \exists s', \eta. s \longrightarrow s'; \eta$	progress predicate
$\text{Shape}_{\Sigma}(v, \tau)$	value-shape predicate

Heap Judgment Forms

$\star \vdash H : \Sigma$	heap H agrees with heap type assignment Σ
$\Sigma \vdash v : \tau$	value v has type τ under heap type assignment Σ
$\Sigma \vdash E : \Gamma$	environment E realizes typing context Γ under Σ
$\Sigma \vdash_{\text{st}} s : \tau$	machine state s is store-resolved at result type τ
$\Sigma \vdash_{\text{tr}} \phi$	trace ϕ is well typed with respect to Σ
$\text{Transport}(\Sigma, s, \Sigma', s')$	one-step heap/store transport from (Σ, s) to (Σ', s')

As previously mentioned, the static relation ($\blacktriangleleft$) is intentionally incomplete: many sound surface effects may summarize the same computation, while some computations may be related only to a coarse surface effect by the supplied inductive rules. Concrete computed actions can refine a region-level distinction and may therefore satisfy computed-effect disjointness even when a region-only comparison would be inconclusive. ‘Top’, by contrast, is a sound approximation of every trace but cannot provide useful disjointness evidence. The current calculus contains no separate static recovery rule for ‘Top’; its role in the mechanized semantics is to conservatively lose precision.

For example, the main purpose of designing a type system where the functional type contains a return type for a computational expression and a return type for an effect expression is to be able to summarize any computational application ($\text{capp } e_f e_a$) by its corresponding effect application ($\text{eapp } e_f e_a$). The interesting aspect of assigning the “extended” functional type to the expression e_f is that it may carry a non-empty latent effect for the computational term which can be soundly summarized by the evaluation of the corresponding “surface” effect term. The returned computed effect of $\text{eapp } e_f e_a$ is produced by the function body’s surface-effect expression. If evaluating e_f or e_a may itself perform memory actions, those actions must be covered by the surface-effect term assigned to the whole computation by the correctness judgment. They are not converted into computed effect components after the fact.

Another example is the expression e_b whose reference type is parametrized by a region p . Two different surface effects are proven sound: the first is as much precise as the static effect would be (rule [EFF-GET]) and the second is potentially “much” more precise than the static effect (rule [EFF-GET-CONC]). The type of the argument of `get` is the reference type parametrized by a region variable or a region constant. Given the region p , the rule [EFF-GET] applies by chaining the surface effect of the argument with the static surface term $\widehat{R}(p)$. When evaluated, this term yields a value that is equivalent to a de facto static effect. However, we can precisely summarize a `get` operation using the surface-effect term $\widehat{\text{Read}}(e)$, which directly takes the argument as input in order to evaluate to a concrete memory address inside some allocated region.

The precision of surface-effect terms can be evaluated by analyzing their dynamic semantics. Consider, for example, the rules [S-Abst-Read] and [S-Conc-Write] in Section 4. The first corresponds to the evaluation of the “static version” of the surface-effect term for the `get` operation and the second corresponds to the evaluation of the “concrete version” of the surface-effect term for the `set` operation. The second rule shows that a concrete address, $l@r$, is being evaluated. Now consider the rules for conflict decision presented in Section 2. When concrete allocated addresses are known, it is possible to decide that the effects `Read` and `Write` do not conflict if the addresses are different even though they belong to the same region, allowing more precision (hence the motivating example presented in Fig. 2). However, if only the region is known, the “static version” of the surface effect, `R` and `W`, does not allow the conflict decision at run-time to be more precise than the static analysis.

7 Small-Step Runtime Safety

The continuation machine supports safety statements over finite executions, not only over completed runs. The small-step safety invariant is indexed by a store typing Σ . Along a finite trace, allocation and mutation may change the heap; preservation therefore produces a final store typing Σ' together with the store-transport relation that carries the state, environment, and continuation invariants from the old heap/store pair to the new one. We write $s \longrightarrow^* s'; \phi$ for a finite small-step execution whose transition fragments accumulate to ϕ , $\text{init}(H, E, P, e)$ for the initial machine state, and $\text{done}(H, v)$ for a successful terminal state. The checked-failure state $\text{error}(H)$ is terminal as well. The judgment $\Sigma \vdash_{\text{st}} s : \tau$ abbreviates the store-resolved typed-state invariant, and $\Sigma \vdash_{\text{tr}} \phi$ states that the trace ϕ is well typed with respect to Σ . We write $s \longrightarrow_{\text{chk}}^* s'; \phi$ for the checked finite execution relation: it follows ordinary non-pair steps, treats a failed pair-parallel computed-effect precheck as $\text{error}(H)$ rather than sequential fallback, and uses the same pair-run state updates with checked branch steps. The predicate $\text{NotStuck}(s)$ means that s is terminal (`done` or `error`) or has a next small-step transition. The predicate $\text{Shape}_{\Sigma}(v, \tau)$ records that the run-time form of v matches type τ under store typing Σ . The progress, preservation, and static-effect trace soundness theorems are stated before Theorem 4, because the surface-effect correctness proof depends on that checked machine layer. The remaining results below specialize completed continuation-machine executions.

Theorem 6 (Successful terminal trace determinism). *For any initial state s , if*

$$s \longrightarrow^* \text{done}(H_1, v_1); \phi_1 \quad \text{and} \quad s \longrightarrow^* \text{done}(H_2, v_2); \phi_2,$$

then

$$\phi_1 = \phi_2 \quad \wedge \quad H_1 \equiv H_2 \quad \wedge \quad v_1 = v_2.$$

Theorem 7 (Terminal surface-effect correctness). *Assume H, E, P form a checked run-time context for Γ, Δ . If*

$$\Gamma; \Delta \vdash e \triangleleft \widehat{\Theta}, \quad \text{init}(H, E, P, e) \longrightarrow_{\text{chk}}^* \text{done}(H', v); \phi,$$

and the surface-effect expression terminates with computed effect Θ ,

$$\text{init}(H, E, P, \hat{\Theta}) \longrightarrow^* \text{done}(H_s, \text{eff}(\Theta)); \phi_s,$$

then the computation trace is covered by the computed effect:

$$\phi \sqsubseteq \Theta.$$

Theorem 8 (Structured terminal surface-effect correctness). *Assume H, E, P form a checked run-time context for Γ, Δ . If*

$$\Gamma; \Delta \vdash e \blacktriangleleft \hat{\Theta}, \quad \text{view}(\text{init}(H, E, P, e), \Phi, \text{done}(H', v)),$$

the flattened view is a checked finite execution,

$$\text{init}(H, E, P, e) \longrightarrow_{\text{chk}}^* \text{done}(H', v); \text{flatten}(\Phi),$$

and the surface-effect expression terminates with computed effect Θ and structured trace view Φ_s ,

$$\text{view}(\text{init}(H, E, P, \hat{\Theta}), \Phi_s, \text{done}(H_s, \text{eff}(\Theta))),$$

then

$$\text{flatten}(\Phi) \sqsubseteq \Theta.$$

The structured-trace theorem is the view-level counterpart of checked terminal surface-effect correctness: coverage is defined by flattening the view to the checked machine trace. The pair-parallel scheduler relation uses the same view datatype differently: it keeps the left and right run-branch views as separate projections and normalizes the scheduled run view to $\Phi_1 \parallel \Phi_2$. Write $s \Longrightarrow_{\text{sch}}^{\Phi_L, \Phi_R} s'$ for the scheduled pair-parallel run relation with left and right branch views Φ_L and Φ_R . For trace views, $\text{NoAlloc}(\Phi)$ abbreviates that $\text{flatten}(\Phi)$ contains no allocation actions.

Theorem 9 (Checked pair-parallel scheduler determinism). *Let*

$$e_L^\mu = \text{capp } e_{f1} e_{a1}, \quad e_R^\mu = \text{capp } e_{f2} e_{a2}, \quad e_L^e = \text{eapp } e_{f1} e_{a1}, \quad e_R^e = \text{eapp } e_{f2} e_{a2}.$$

Assume

$$\text{CheckedCtx}_{\Gamma, \Delta}(H, E, P) \quad \text{and} \quad \Gamma; \Delta; P \vdash \langle e_L^\mu, e_R^\mu \rangle \blacktriangleleft e_L^e \oplus e_R^e.$$

Suppose the two surface-effect applications terminate with computed effects Θ_L and Θ_R and the computed-effect precheck succeeds:

$$\begin{aligned} \text{init}(H, E, P, e_L^e) \longrightarrow^* \text{done}(H_L^e, \text{eff}(\Theta_L)); \phi_L^e, \quad \text{init}(H, E, P, e_R^e) \longrightarrow^* \text{done}(H_R^e, \text{eff}(\Theta_R)); \phi_R^e, \\ \neg(\Theta_L \otimes \Theta_R). \end{aligned}$$

Let

$$s_0 = \text{pairRun}(\text{init}(H, E, P, e_L^\mu), \text{init}(H, E, P, e_R^\mu), [], [], k).$$

For $j \in \{1, 2\}$, assume the scheduled branch views are covered by the corresponding computed effects and contain no allocations,

$$\Phi_L^j \sqsubseteq \Theta_L \quad \text{and} \quad \Phi_R^j \sqsubseteq \Theta_R, \quad \text{and} \quad \text{NoAlloc}(\Phi_L^j) \quad \text{and} \quad \text{NoAlloc}(\Phi_R^j),$$

and that both scheduled executions successfully join:

$$s_0 \Longrightarrow_{\text{sch}}^{\Phi_L^1, \Phi_R^1} \langle H_1, (v_L^1, v_R^1), k \rangle \quad \text{and} \quad s_0 \Longrightarrow_{\text{sch}}^{\Phi_L^2, \Phi_R^2} \langle H_2, (v_L^2, v_R^2), k \rangle.$$

Then

$$H_1 \equiv H_2 \quad \wedge \quad v_L^1 = v_L^2 \quad \wedge \quad v_R^1 = v_R^2.$$

The no-allocation assumptions used for the run branches are exactly the $\text{NoAlloc}(\epsilon_1[P])$ and $\text{NoAlloc}(\epsilon_2[P])$ premises in the pair-parallel correctness rule. They are weaker than heap-neutrality: run branches may write, while surface-effect evaluation is the part required to be heap-neutral.

For a list trace ϕ , write

$$\text{Wr}(\phi) = \{ l@r \mid \text{Write}(l@r) \in \phi \}$$

for its heap-write footprint. If W is a set of heap addresses, write $H_1 \equiv_{\overline{W}} H_2$ when the two heaps agree outside W :

$$H_1 \equiv_{\overline{W}} H_2 \iff \forall a. a \notin W \Rightarrow H_1(a) = H_2(a).$$

Theorem 10 (Checked scheduler error-cause determinism). *Under the assumptions of the scheduler determinism theorem, suppose two derivations of the same scheduled relation terminate in checked failure:*

$$s_0 \Rightarrow_{\text{sch}}^{\Phi_L^1, \Phi_R^1} \text{error}(H_1) \quad \text{and} \quad s_0 \Rightarrow_{\text{sch}}^{\Phi_L^2, \Phi_R^2} \text{error}(H_2).$$

Then the two derivations have the same branch error cause. Either both fail because the left branch reaches an error,

$$s_0 \Rightarrow_{\text{sch}}^{\Phi_L^i, \Phi_R^i} \text{pairRun}(\text{error}(H_i), s_R^i, \phi_L^i, \phi_R^i, k) \quad (i = 1, 2),$$

and, writing

$$W_R^i = \text{Wr}(\text{flatten}(\Phi_R^i)),$$

the final error heaps agree outside the addresses written by either right projection,

$$H_1 \equiv_{\overline{W_R^1 \cup W_R^2}} H_2;$$

or both fail because the right branch reaches an error after the left branch has completed,

$$s_0 \Rightarrow_{\text{sch}}^{\Phi_L^i, \Phi_R^i} \text{pairRun}(\text{done}(H_L^i, v_L^i), \text{error}(H_i), \phi_L^i, \phi_R^i, k) \quad (i = 1, 2),$$

and in that case

$$H_1 \equiv H_2.$$

The mixed cases, where one derivation is left-caused and the other is right-caused, are impossible. In particular, if both right projections perform no writes, then the left-caused case gives global heap-lookup agreement.

Corollary 3 (Scheduled continuation determinism). *Under the assumptions of the scheduler determinism theorem, if the two joined states continue to successful terminal states,*

$$\langle H_1, (v_L^1, v_R^1), k \rangle \longrightarrow^* \text{done}(H'_1, w_1); \psi_1 \quad \text{and} \quad \langle H_2, (v_L^2, v_R^2), k \rangle \longrightarrow^* \text{done}(H'_2, w_2); \psi_2,$$

then

$$\psi_1 = \psi_2 \quad \wedge \quad H'_1 \equiv H'_2 \quad \wedge \quad w_1 = w_2.$$

Corollary 4 (Left-to-right run embedding). *Let*

$$e_L^\mu = \text{capp } e_{f1} e_{a1} \quad \text{and} \quad e_R^\mu = \text{capp } e_{f2} e_{a2}.$$

For

$$s_0 = \text{pairRun}(\text{init}(H, E, P, e_L^\mu), \text{init}(H, E, P, e_R^\mu), [], [], \text{doneK}),$$

if the ordinary left-to-right machine execution terminates successfully,

$$s_0 \longrightarrow^* \text{done}(H', v); \phi,$$

then there exist branch views Φ_L, Φ_R such that

$$s_0 \Rightarrow_{\text{sch}}^{\Phi_L, \Phi_R} \langle H', v, \text{doneK} \rangle \quad \text{and} \quad \phi \equiv_{\text{perm}} \text{flatten}(\Phi_L \parallel \Phi_R).$$

8 Discussion and Related Work

The closest static comparator is Deterministic Parallel Java (DPJ) (Bocchino et al., 2009). DPJ uses a modular compile-time type-and-effect system to guarantee deterministic semantics and expresses effects over regions. Surface effects retain static verification but move a value-dependent access description into an executable source-level term. From the run-time side, classic inspector-executor work analyzes irregular dependences before the executor runs conflict-free work in parallel (Arenaz et al., 2004). The distinctive point of the present formulation is therefore not region effects or pre-execution access prediction in isolation, but the integration of a programmer-written, typed surface-effect term with an approximation theorem and a trace-composition proof.

This positioning also exposes a limitation. Expressive region/effect annotations can be difficult to write manually, and prior work has investigated inference precisely to reduce that burden (Tzannes et al., 2019). The present calculus assumes programmer-written surface effects and does not give an inference or synthesis algorithm. Annotation effort and the frequency with which surface effects collapse to ‘Top’ are empirical questions left open here.

Hybrid verification platforms are also close in spirit (Kawaguchi et al., 2012; Flanagan et al., 2006). They use expressive specifications, such as refinement types, to combine static reasoning with value-sensitive properties. Liquid Types support deterministic fine-grained multithreading by expressing effects in first-order formulas over a decidable arithmetic theory. At a higher-level object-oriented setting, HOOL supports expressive object specifications, including subtyping, refinements, and object invariants. As in our calculus, these specifications are pure: evaluating them does not update mutable data.

The design is hybrid in a precise sense. Verification of the computation/surface-effect relation is static, while the computed effect is dynamic: the surface-effect term is evaluated with run-time values and its computed effect can be checked for disjointness and conflict absence by later clients. Surface-effect evaluation is still run-time computation and may itself be input dependent. The continuation-machine metatheory proves progress, preservation, terminal determinism, and terminal surface-effect correctness for checked executions. It does not prove termination of surface-effect terms, bound their cost, or compare that cost with dynamic validation mechanisms. A future adequacy theorem could relate terminal machine runs to a terminating evaluator, but that bridge is not used as a paper theorem here.

Recent work on hybrid type and effect systems applied to concurrent programming is found in Long et al. (2013). The approach defines special static effects, designated open effects, that prevent object-oriented programs from being rejected when no static upper bound in the form of effect specification is available. More specifically, open effects are assumed not to conflict with any other effects and are used when the dynamic type of receiver methods cannot be known statically due to dynamically dispatched method invocation. Although open effects are somewhat analogous to the surface-effects approach, in the sense that disjointness information may be unknown until runtime, there is a subtle difference: while open effects are inductively defined and designed for optimistic parallelism, where concretized effects soundly approximate runtime effects, surface-effect terms are syntactically defined in terms of unevaluated expressions and their concretization derives from runtime heap allocation. The correctness of surface-effect terms is always statically known, and the purpose of dynamic checking is to find further possibilities for parallelism. Moreover, surface-effect terms can be used in those scenarios where conflict decision based on static effects would reject parallel programs. Since more precise effect specifications, yet statically verified, are provided directly by the programmer, surface-effect analysis can override static-effect analysis.

Irregular data structures such as trees, queues, and graphs motivate the use of run-time information because region-level analyses may merge accesses that are concrete-address disjoint. The Galois approach (Kulkarni et al., 2007) is a useful contrast because it uses run-time conflict detection in an optimistic execution model. The surface-effects approach instead evaluates a surface effect before the parallel computation and therefore does not require the calculus to roll back a computation after a surface-effect conflict is found. This is a semantic difference, not an established performance comparison.

The present work supplies neither a cost semantics nor an implementation study. Consequently, we make no claim that surface-effect evaluation is constant in the size of the surface-effect term, that pre-checking has lower total overhead than access logging, or that useful parallelism is recovered frequently in practice. A future evaluation should measure at least surface-effect-evaluation time,

conflict-test time, false-positive rate, the frequency of ‘Top’, recovered parallelism relative to region-only static effects, and programmer annotation effort. These measurements are necessary to determine whether the formal precision mechanism translates into a practical advantage.

9 Conclusions

We have presented a region-polymorphic calculus in which a computation is summarized by a statically verified, executable surface-effect term. Evaluating the surface effect before the computation can expose concrete region/location actions that a region-only static effect does not distinguish. The mechanized correctness theorem proves trace approximation, and the trace-composition lemma makes the bridge explicit: disjoint and non-conflicting computed effects for deterministic component traces yield a deterministic parallel trace. The trace semantics then establishes equivalent terminating replay heaps. The continuation-machine theorem stack adds checked runtime safety: well-typed states make progress, machine steps preserve typing, and two terminal executions from the same initial state have the same trace, heap, and value. For checked pair-parallel run phases, the scheduled-run theorem shows that successful joins have the same heap and pair value for all branch interleavings represented by the scheduler views, with the ordinary left-then-right run embedded as a scheduler instance.

These results are formal, not empirical performance results. The calculus does not include surface-effect inference, a termination argument for surface-effect evaluation, a cost semantics, an implementation evaluation, or a separate adequacy theorem for terminal continuation-machine runs. Establishing the annotation burden and the run-time trade-off between surface-effect precision, pre-checking cost, and recovered parallelism is therefore left to future work.

References

- Manuel Arenaz, Juan Touriño, and Ramón Doallo. An inspector-executor algorithm for irregular assignment parallelization. In *Parallel and Distributed Processing and Applications*, volume 3358 of *Lecture Notes in Computer Science*, pages 4–15. Springer, 2004. doi: 10.1007/978-3-540-30566-8_4.
- Robert L. Bocchino, Jr., Vikram S. Adve, Danny Dig, Sarita V. Adve, Stephen Heumann, Rakesh Komuravelli, Jeffrey Overbey, Patrick Simmons, Hyojin Sung, and Mohsen Vakilian. A type and effect system for deterministic parallel Java. *SIGPLAN Not.*, 44(10):97–116, 2009.
- Arthur Charguéraud. The locally nameless representation. *Journal of Automated Reasoning*, 49(3):363–408, 2012.
- Cormac Flanagan, Stephen N. Freund, and Aaron Tomb. Hybrid types, invariants, and refinements for imperative objects. In *International Workshop on Foundations and Developments of Object-Oriented Languages (FOOL)*, 2006.
- Ming Kawaguchi, Patrick Rondon, Alexander Bakst, and Ranjit Jhala. Deterministic parallelism via liquid effects. *SIGPLAN Not.*, 47(6):45–54, 2012.
- Milind Kulkarni, Keshav Pingali, Bruce Walter, Ganesh Ramanarayanan, Kavita Bala, and L. Paul Chew. Optimistic parallelism requires abstractions. *SIGPLAN Not.*, 42(6):211–222, 2007.
- Yuheng Long, Mehdi Bagherzadeh, and Hriday Rajan. Open effects: A hybrid type-and-effect system to tackle the open world assumption and its application to optimistic concurrency. Technical Report TR13-04a, Iowa State University, 2013.
- John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’88)*, pages 47–57, New York, NY, USA, 1988. ACM.
- Alexandros Tzannes, Stephen T. Heumann, Lamya Eloussi, Mohsen Vakilian, Vikram S. Adve, and Michael Han. Region and effect inference for safe parallelism. *Automated Software Engineering*, 26(2):463–509, 2019. doi: 10.1007/s10515-019-00257-3.

A Deterministic Parallelism

This appendix records the replay-oriented deterministic-parallelism layer used by the earlier trace semantics. It is separate from the primitive continuation-machine trace in Section 4: the machine trace there is a list of value-free dynamic actions, while this replay layer uses the richer trace syntax needed to state replay over heaps.

Definition 1 (Heap equivalence). *The mechanization represents a heap as a finite map from keys (r, l) to values. We write $H_1 \equiv H_2$ when the two finite maps are extensionally equal, equivalently when every key has the same lookup result in both heaps:*

$$H_1 \equiv H_2 \iff \forall k. H_1(k) = H_2(k).$$

This is equality of key-value maps, not an isomorphism modulo renaming of locations. The relation is reflexive, symmetric, and transitive.

Definition 2 (Deterministic trace). *The predicate $\text{Det}(\phi)$ is inductively generated by the empty trace, single dynamic actions, sequential composition of deterministic traces, and a parallel constructor. The latter requires deterministic component traces and both*

$$\neg \text{ConflictTraces}(\phi_1, \phi_2) \quad \text{and} \quad \text{DisjointTraces}(\phi_1, \phi_2).$$

Thus absence of an explicitly represented read-write or write-write conflict is not the only premise: every pair of dynamic actions drawn from opposite branches must also satisfy dynamic-action disjointness.

Definition 3 (Trace replay). *The relation $\Rightarrow$ executes one dynamic action of a trace against a heap. Allocation and write actions update the heap, reads require the recorded value to be present, and sequential/parallel trace nodes provide the evaluation contexts. We write $\Rightarrow^*$ for its reflexive transitive closure. A terminating replay has the form $(\phi, H) \Rightarrow^* (\text{Nil}, H')$.*

Lemma 3 (Read-trace properties). *A trace consisting only of reads preserves the heap during replay. Two such component traces form a deterministic parallel trace. These results follow from the inductive trace predicate whose constructors contain only the empty trace, read actions, and sequential/parallel compositions.*

Lemma 4 (Scheduler soundness). *For $i \in \{1, 2\}$, let $\phi_i \sqsubseteq \Theta_i$ and $\text{Det}(\phi_i)$. If*

$$\text{Disjoint}(\Theta_1, \Theta_2) \quad \wedge \quad \neg(\Theta_1 \otimes \Theta_2),$$

then $\text{Det}(\phi_1 \parallel \phi_2)$.

The proof uses the soundness relation $\sqsubseteq$ in both directions required by the parallel-trace constructor: a dynamic conflict would imply a computed conflict, and computed-action disjointness implies disjointness of every represented pair of dynamic actions.

Theorem 11 (Confluence of deterministic trace replay). *Let $H_a \equiv H_b$. If a deterministic trace ϕ is replayed from the two equivalent heaps to intermediate configurations (ϕ_1, H_1) and (ϕ_2, H_2) , then there exist a common trace ϕ_3 and heaps $H_3 \equiv H_4$ such that*

$$(\phi_1, H_1) \Rightarrow^* (\phi_3, H_3) \quad \text{and} \quad (\phi_2, H_2) \Rightarrow^* (\phi_3, H_4).$$

In particular, if both replays terminate at Nil, their final heaps are equivalent.

The result does not claim uniqueness of an intermediate trace independently of the stated confluence property.

Theorem 12 (Heap uniqueness for safe parallel surface effects). *Let $\phi_1 \sqsubseteq \Theta_1$ and $\phi_2 \sqsubseteq \Theta_2$, with deterministic component traces and*

$$\text{Disjoint}(\Theta_1, \Theta_2) \quad \wedge \quad \neg(\Theta_1 \otimes \Theta_2).$$

Let $H_a \equiv H_b$. If

$$(\phi_1 \parallel \phi_2, H_a) \Rightarrow^* (\text{Nil}, H_1) \quad \text{and} \quad (\phi_1 \parallel \phi_2, H_b) \Rightarrow^* (\text{Nil}, H_2),$$

then $H_1 \equiv H_2$.

The proof applies trace-composition soundness to establish determinism of the parallel trace and then uses the terminating replay theorem.